Methodology

In this chapter, we describe the algorithm used to construct, search, and recover structured sparse Transformer architectures under a fixed MAC budget. Given that this constraint preserves only a small proportion of the original model, it is necessary for the method to choose which important layers, attention heads, and feed-forward neurons to keep. Hence, the quality of the final model depends on three steps: defining the sparse architecture, comparing candidates during the search, and recovering the selected model after pruning.

The chapter follows this order. The first section establishes the binary mask parameterisation of the units to be searched, namely layers, attention heads, and feed-forward neurons. The resulting search space contains candidates distinguished by the distribution of these units throughout the model and by their overall sparsity ratio. The next section introduces the cost model and the hard feasibility band that limits the search to candidates with similar MAC requirements. Once the total cost is constrained, the main decision becomes how the available computation should be allocated within the model.

Nonetheless, the problem with the allocation alone is that it is hard to differentiate between one architecture and another. This leads to the use of a ranking metric that can compare many unrecovered candidates without fully fine-tuning each one of them. The metric shall be designed to evaluate how well a sparse architecture preserves the teacher's internal computation before any form of recovery, while still retaining task signals using a fixed calibration data.

With the search space, the budget model, the ranking metric, and the calibration data all defined, this chapter focuses on the search algorithm. It then proposes an abstract definition of this procedure as a constrained evolutionary algorithm involving proposal scoring, structure-aware initialization, regional populations, and a stepped evaluation funnel, aiming to discover numerous possible architectural assignments while maintaining diversity across qualitatively different sparse shapes. The detailed pipeline section then presents the search procedure step by step, describing how the proposal priors are transformed into candidate masks and how these masks are progressively evaluated, selected, and mutated.

The last section of the chapter covers recovery. The architecture found by the search is merely a sparse mask and not yet the fully adapted compressed model; the subsequent recovery procedure thus is the mechanism to fine-tune the survivors according to the given architecture. The recovery procedure is done in two stages: the recovery of the intermediate representation helps to stabilize the internally masked network, while prediction level distillation forces adaptation at the level of task output.

Combined, the above sections form a single, coherent pipeline: the parameterisation of binary structural parameters, the search subject to a budget, ranking of candidate designs via surrogates, architecture selection via evolution, and finally post-search recovery using a fixed mask. As such, the aim of the chapter is twofold; to define the necessary formal objects for a precise statement of the method and to define the necessary algorithmic steps to achieve an end-to-end implementation of the method.

1.1 Mask Parameterisation

This section defines the architecture variable optimised by the search. Each candidate is represented by two binary masks: one over attention heads and one over feed forward neurons. The masks do not change the embedding dimension, classifier interface, residual topology, or layer ordering. They only decide which structured units remain active, matching the unit level used by structured transformer pruning methods (Michel et al., 2019; Xia et al., 2022; Kurtic et al., 2023).

1.1.1 Candidate Space

A candidate architecture is a pair of binary masks

$$c = (\mathbf{M}^H, \mathbf{M}^N) \in \{0,1\}^{L\times H} \times \{0,1\}^{L\times N}. \tag{1.1}$$

For layer ℓ, the active attention heads and active feed forward neurons are

$$H_\ell(c) = \{h : \mathbf{M}^H[\ell, h] = 1\}, \qquad N_\ell(c) = \{n : \mathbf{M}^N[\ell, n] = 1\}. \tag{1.2}$$

The head mask selects complete head slices in the attention projections. The neuron mask selects rows of the first feed forward projection and the matching columns of the second feed forward projection. The masks are indexed in the original teacher coordinates.

The remaining definitions are derived from this same object: structural counts, layer status, and candidate identity.

1.1.2 Structural Counts

The first derived object is the count vector induced by the two masks. The count vector summarizes the mask for budget control and reporting. At layer ℓ, summing the binary decisions in each row gives

$$a_\ell(c) = \sum_{h=1}^{H} \mathbf{M}^H[\ell, h], \tag{1.3}$$

$$n_\ell(c) = \sum_{n=1}^{N} \mathbf{M}^N[\ell, n]. \tag{1.4}$$

Thus $a_\ell(c)$ records how many attention heads remain in layer ℓ, whereas $n_\ell(c)$ records how many feed forward neurons remain in that same layer. The search keeps these local quantities because cost and behaviour depend on where units are kept.

The global head and neuron counts are then obtained by summing the local counts across depth:

$$A(c) = \sum_{\ell=1}^{L} a_\ell(c), \qquad N(c) = \sum_{\ell=1}^{L} n_\ell(c). \tag{1.5}$$

These totals report candidate size. They do not show whether the remaining units are concentrated in a few layers or distributed across many layers. Therefore, we also track the number of layers that contain at least one active sublayer:

$$L_{\text{act}}(c) = \sum_{\ell=1}^{L} \mathbf{1}\{a_\ell(c) + n_\ell(c) > 0\}. \tag{1.6}$$

Finally, the same per-layer counts are used to evaluate the compute budget. Each active head contributes one head cost, and each active feed forward neuron contributes one neuron cost:

$$\mathrm{MAC}(c) = \sum_{\ell=1}^{L} \big(a_\ell(c)c_H + n_\ell(c)c_N\big), \tag{1.7}$$

where c_H and c_N are the per head and per neuron costs derived in Section 1.2.

The pair (a_ℓ, n_ℓ) also fixes the execution case of layer ℓ. Table 1.1 shows how these counts determine the masked forward pass. The search therefore selects both the budget and the computation that remains at each depth.

Condition	Status	Masked forward
$a_\ell > 0,\ n_\ell > 0$	both active	attention and feed forward execute
$a_\ell = 0,\ n_\ell > 0$	feed forward only	attention update is zero; feed forward executes
$a_\ell > 0,\ n_\ell = 0$	attention only	attention executes; feed forward update is zero
$a_\ell = 0,\ n_\ell = 0$	identity	the layer returns its input

Table 1.1: Layer statuses induced by the structural masks.

This status is used whenever a candidate is evaluated. Positive counts keep the corresponding sublayer active, whereas zero counts remove that sublayer update. Therefore, two candidates with the same global size can behave differently if their active units create different layer statuses. The implementation also needs two levels of candidate description: an exact identity for caching and a coarser signature for population tracking.

1.1.3 Granularity and Candidate Identity

Neuron choices are grouped for search efficiency. The implementation allocates feed forward neurons in blocks of sixteen. Thus $n_\ell(c)$ is restricted to multiples of sixteen, except for the zero case. This block size keeps mutation aligned with meaningful cost changes and reduces candidates that differ only by negligible single-neuron moves.

For exact caching, the identity of a candidate is the bit string obtained by concatenating $\mathbf{M}^H$ and $\mathbf{M}^N$. Two candidates are identical if and only if this bit string is identical. This key is deliberately fine-grained because it preserves the exact placement of every active unit. For diversity tracking, the search also records the coarser macro signature

$$\sigma(c) = \big(A(c),\, N(c),\, L_{\mathrm{act}}(c)\big). \tag{1.8}$$

The macro signature loses placement information. Two candidates can have the same total counts and active layer count while assigning compute to different depths, so exact deduplication still uses the full bit string. The macro signature is used only as a population-level descriptor in the search mechanisms described in Section 1.6.

1.2 Efficiency Budget and Cost Model

The binary masks of Section 1.1 determine the active attention heads and feed-forward neurons in each layer. The search therefore assigns a deterministic operation cost to every candidate and uses this cost for feasibility checks, repair, and reporting. All costs are evaluated at the reference sequence length S.

For one active attention head with head dimension d_h, the mask-dependent projection cost is

$$c_H = 2Sdd_h. \tag{1.9}$$

For one active feed-forward neuron, the corresponding cost of the two linear projections is

$$c_N = 2Sd. \tag{1.10}$$

Let $a_\ell(c)$ and $n_\ell(c)$ denote the active head and neuron counts of layer ℓ. The layer and candidate costs are

$$\mathrm{MAC}_\ell(c) = a_\ell(c)c_H + n_\ell(c)c_N, \tag{1.11}$$

$$\mathrm{MAC}(c) = \sum_{\ell=1}^{L} \mathrm{MAC}_\ell(c). \tag{1.12}$$

The dense reference cost is

$$\mathrm{MAC}_{\mathrm{dense}} = L(Hc_H + Nc_N),$$

and the keep ratio is $\rho(c) = \mathrm{MAC}(c)/\mathrm{MAC}_{\mathrm{dense}}$.

The target budget B is discrete because head counts are integer and neuron counts are allocated in blocks. A candidate is feasible when its cost lies in the tolerance band

$$(1-\varepsilon)B \le \mathrm{MAC}(c) \le (1+\varepsilon)B. \tag{1.13}$$

This constraint fixes the candidate scale before search. The remaining selection problem is the allocation of the allowed compute across depth, attention heads, and feed-forward neurons.

1.3 The Surrogate Distance Metric

After budget filtering, the candidates being scored are already close in operation count. The remaining question is allocation. The same budget can be placed at different depths and split differently across the two structural axes. A useful search metric therefore has to compare how these allocations behave, since the cost alone no longer separates them.

The metric is evaluated before recovery training. This restriction is essential because the search scores thousands of masks, and running a full distillation or fine tuning procedure for every candidate would make the search infeasible. The metric therefore has to judge an unrecovered masked model from a small calibration pack. Validation accuracy and final task loss are weak by themselves at this stage because high-sparsity candidates often produce flat or collapsed outputs.

The implementation uses two ranking steps. The first step ranks candidates by residual-update preservation and keeps the best fraction of the cohort. The second step ranks the survivors by a task-facing distance. This order reflects the role of the metric in the search: a candidate must first preserve a plausible computation path, then the task signal decides which preserved path is already closest to the teacher's prediction surface.

1.3.1 Equal-Cost Candidate Ranking

The budget constraint turns a large architecture space into a local comparison problem. The metric receives a cohort of feasible allocations near the target cost, so the candidates differ mainly in where their remaining units are placed.

Let $\mathcal{C}_t$ be the cohort evaluated at generation t. Every $c \in \mathcal{C}_t$ satisfies the feasibility band

$$(1-\varepsilon)B \le \mathrm{MAC}(c) \le (1+\varepsilon)B. \tag{1.14}$$

The band makes the members of $\mathcal{C}_t$ comparable in cost. Small residual MAC differences inside the band are treated as secondary; the main comparison is the placement of the retained heads and feed forward neurons. This is why the score is cohort-local: it decides which feasible allocation should survive this generation.

For each candidate, the teacher $\mathcal{T}$ and masked student $\mathcal{S}(c)$ are evaluated on a calibration pack

$$\mathcal{D}_{\text{cal}} = \{x_i\}_{i=1}^{B}.$$

Let $p_{i,s} \in \{0, 1\}$ indicate whether token position s of example i is non-padding. All hidden-state and update distances are computed only over positions where $p_{i,s} = 1$.

The output of the metric is a cohort-local ordering. Lower values are better:

$$\Phi(c_1) < \Phi(c_2) \quad \Longrightarrow \quad c_1 \text{ is preferred to } c_2 \text{ inside the current search generation.}$$

This ordering drives search-time selection before recovery. The first quantity needed by the two-step rule is the structural rank, so the metric starts from the layer updates.

1.3.2 Residual Update Matching

The structural rank is based on a simple intuition: a retained architecture should reproduce the transformations made along the teacher's depth. Comparing final hidden states is too coarse for this purpose. A hidden state contains the accumulated residual stream, so a weak layer can be hidden by computation that happened earlier. The layer update exposes the contribution made at one depth.

Let h_ℓ denote the hidden state after layer ℓ, with h_0 the embedding output. The layer update is

$$\Delta h_\ell = h_\ell - h_{\ell-1}. \tag{1.15}$$

For the teacher and student, these updates are written as $\Delta_\ell^{(\mathcal{T})}$ and $\Delta_\ell^{(\mathcal{S})}$.

The update Δh_ℓ isolates the computation performed at layer ℓ. If a masked layer contributes almost nothing, then $\Delta_\ell^{(\mathcal{S})}$ becomes small even when the absolute hidden state remains numerically close to the teacher stream. This makes update matching a better search signal for partial and fully dropped layers.

For tensors A and B shaped as batch, sequence, and hidden dimension, define the calibration inner product

$$\langle A,\, B\rangle_{\text{cal}} = \sum_{i=1}^{B}\sum_{s=1}^{S} p_{i,s} A_{i,s}^{\top} B_{i,s}, \qquad \|A\|_{\text{cal}} = \sqrt{\langle A,\, A\rangle_{\text{cal}}}. \tag{1.16}$$

The inner product sums only real token positions, so padding does not change the distance. The per-layer trajectory distance then checks two properties of the student update. Direction agreement asks whether the student update points the same way as the teacher update. Projection agreement asks whether the student recovers the teacher update with the right scale along that direction:

$$d_{\ell,\cos}^{\text{traj}}(c) = \frac{1}{\pi} \arccos\left(\frac{\left\langle \Delta_\ell^{(\mathcal{S})},\, \Delta_\ell^{(\mathcal{T})} \right\rangle_{\text{cal}}}{\left\| \Delta_\ell^{(\mathcal{S})} \right\|_{\text{cal}} \left\| \Delta_\ell^{(\mathcal{T})} \right\|_{\text{cal}} + \delta} \right), \tag{1.17}$$

$$\alpha_\ell(c) = \frac{\left\langle \Delta_\ell^{(\mathcal{S})},\, \Delta_\ell^{(\mathcal{T})} \right\rangle_{\text{cal}}}{\left\| \Delta_\ell^{(\mathcal{T})} \right\|_{\text{cal}}^2 + \delta}, \tag{1.18}$$

$$d_{\ell,\text{proj}}^{\text{traj}}(c) = \left| 1 - \alpha_\ell(c) \right|, \tag{1.19}$$

$$d_\ell^{\text{traj}}(c) = (1-\lambda) d_{\ell,\cos}^{\text{traj}}(c) + \lambda d_{\ell,\text{proj}}^{\text{traj}}(c). \tag{1.20}$$

The constant δ is a numerical stabiliser. The cosine term measures direction agreement between the student and teacher updates. The projection coefficient α_ℓ measures how much of the teacher update is recovered by the student along the teacher direction. A value near one means the

student update has the correct magnitude in the teacher direction. A value near zero means the update is missing or orthogonal. Values above one mean the student overshoots the teacher update.

Both parts are needed. A layer with the right norm and wrong direction fails the cosine term. A layer with the right direction and weak magnitude fails the projection term. A dropped layer gives $\Delta_\ell^{(\mathcal{S})} \approx 0$, so its projection coefficient is close to zero. The per-layer distance therefore penalises missing, misdirected, and over-scaled computation.

1.3.3 Aggregation Across Layers

A candidate is selected as a whole architecture, so the layer distances need a single structural score. The aggregation below turns the sequence of per-layer errors into one trajectory distance:

$$d^{\mathrm{traj}}(c) = \frac{1}{Z} \sum_{\ell=1}^{L} w_\ell d_\ell^{\mathrm{traj}}(c), \qquad Z = \sum_{\ell=1}^{L} w_\ell. \tag{1.21}$$

The weights w_ℓ determine how depth contributes to the structural score. The simplest choice is $w_\ell = 1$, which gives each layer equal influence. A magnitude-weighted variant uses a function of $\left\| \Delta_\ell^{(\mathcal{T})} \right\|_{\mathrm{cal}}$, so layers where the teacher update is larger receive more weight. A late-layer-weighted variant can emphasise depths closer to the classifier. These choices change the search pressure, so the experiments report them as metric ablations.

The trajectory score is the first-stage rejector in the metric. Candidates with many missing, misdirected, or scale-wrong updates receive high d^{traj}. Candidates that preserve a coherent computation path receive lower d^{traj}. At this point the metric has identified which masks still perform a plausible internal computation; the next stage can then use final-output evidence to order those masks.

1.3.4 Task-Facing Distance After Structural Filtering

The structural score does not inspect the final label decision. The second ranking stage fills that gap with task-facing evidence from the final logits. This signal is useful after the update gate, because the surviving candidates have already preserved enough internal computation to make their logits meaningful.

Let $z_i^{(\mathcal{T})}$ and $z_i^{(\mathcal{S})}(c)$ be the teacher and student logits for input x_i. The distillation distance is

$$d^{\mathrm{kd}}(c) = T^2 \frac{1}{B} \sum_{i=1}^{B} \mathrm{KL}\Big(\mathrm{softmax}(z_i^{(\mathcal{T})}/T) \,\Big\|\, \mathrm{softmax}(z_i^{(\mathcal{S})}(c)/T) \Big), \tag{1.22}$$

where T is the distillation temperature. The hard task-facing term is

$$d^{\mathrm{ce}}(c) = -\frac{1}{B} \sum_{i=1}^{B} \log \mathrm{softmax}(z_i^{(\mathcal{S})}(c))_{y_i^\star}, \tag{1.23}$$

where $y_i^\star$ is either the true label or the teacher argmax label, depending on the calibration mode.

At high sparsity, many unrecovered candidates produce logits that are nearly uniform, biased toward one class, or unstable under small mask changes. In that regime, d^{kd} and d^{ce} can be similar for masks with different internal computations. This is the reason the search does not use the task distance as the first gate. The task distance is written as

$$d^{\mathrm{task}}(c) = \beta_{\mathrm{kd}} d^{\mathrm{kd}}(c) + \beta_{\mathrm{ce}} d^{\mathrm{ce}}(c), \tag{1.24}$$

with non-negative coefficients. This distance is computed for every candidate in the cohort, then converted to a percentile rank. The final selection rule uses this task rank after the update-rank gate has selected the structurally acceptable candidates.

A low task distance from a collapsed candidate can occur for accidental reasons on a small calibration pack. A low task distance from a candidate with good residual updates is more meaningful, since the candidate has preserved a computation path capable of supporting recovery.

1.3.5 Two-Step Cohort Ranking

The search implementation uses the double-rank gate in `updt_gate` mode. This is the point where the two distances become the actual selection score. Raw trajectory and task distances have different units, ranges, and noise levels, so the implementation compares candidates by percentile ranks within the current cohort.

For any component d, define the lower-is-better percentile rank

$$r_d(c) = \frac{1}{\max(|\mathcal{C}_t| - 1, 1)} \operatorname{rank}_{\mathcal{C}_t}(d(c)), \tag{1.25}$$

where ties receive their averaged rank and lower values are better. The two ranks used by the search are

$$r^{\mathrm{traj}}(c) = r_{d^{\mathrm{traj}}}(c), \qquad r^{\mathrm{task}}(c) = r_{d^{\mathrm{task}}}(c).$$

Here r^{traj} comes from the trajectory distance and r^{task} comes from the task-facing distance. The ranking rule then uses them in a fixed order.

The first ranking step keeps candidates whose update rank is inside the retained fraction ρ:

$$\mathcal{S}_t = \{c \in \mathcal{C}_t : r^{\mathrm{traj}}(c) \le \rho\}, \qquad \rho = \texttt{UPDT\_GATE\_KEEP\_FRAC}.$$

This step implements the structural filter: candidates outside $\mathcal{S}_t$ have weaker update preservation than the retained fraction of the cohort. The second step orders the survivors by task rank, with a small update-rank tie-break:

$$\Phi(c) = \begin{cases} r^{\mathrm{task}}(c) + \epsilon r^{\mathrm{traj}}(c), & c \in \mathcal{S}_t, \\ 1 + r^{\mathrm{traj}}(c), & c \notin \mathcal{S}_t. \end{cases} \tag{1.26}$$

The first branch is the survivor ordering used for selection. The second branch assigns every rejected candidate a score above one, placing it behind the survivor set. Rejected candidates still keep their update-rank order, so the genetic search retains pressure toward better structure even among candidates that fail the current gate.

1.3.6 Calibration Noise and Resampling

The two-step ranking depends on measurements from a finite calibration pack. A candidate near the update gate can change rank if the pack changes. The implementation reduces this variance by averaging component distances over independent packs before computing ranks. Let $\mathcal{P}_1, \ldots, \mathcal{P}_R$ be independently sampled calibration packs. A resampled component is computed as

$$\bar{d}(c) = \frac{1}{R} \sum_{r=1}^{R} d_{\mathcal{P}_r}(c). \tag{1.27}$$

The same averaging applies to d^{traj}, d^{kd}, and d^{ce} before cohort ranks are computed.

The search uses two fidelity levels. Cheap evaluation scores many candidates on a small pack and drives broad exploration. Full evaluation averages over larger or multiple packs for candidates that may survive, enter the Hall of Fame, or be compared across generations. The cheap score gives speed. The full score reduces rank flips near the structural gate.

Resampling is especially important around κ. A small calibration perturbation can move a candidate from survivor to non-survivor. Averaging reduces these boundary errors and makes the selected population less dependent on one calibration draw.

1.3.7 Limits of the Surrogate

The surrogate ranks unrecovered candidates. It cannot guarantee recovered validation accuracy. Recovery can repair a candidate with a mediocre initial surrogate score, and recovery can fail on a candidate whose initial updates look good. The metric is therefore evaluated empirically by rank correlation with post-recovery validation accuracy in the later diagnostic experiments.

1.4 Calibration Data and Signal Collection

The search metric in Section 1.3 is evaluated many times, so it cannot use the full training or validation set for every candidate. It uses a small calibration subset instead. This subset supplies the inputs on which the teacher and the masked student are compared during search.

1.4.1 Calibration Subset

The calibration subset is sampled from the training split with a fixed seed. The validation split is reserved for reporting recovered accuracy, so it is not used as the search signal. This keeps the search target separate from the final evaluation target.

Each calibration example is tokenised in the same way as the task data used for training and recovery. Padding positions are tracked, because hidden-state and update distances are computed only on real tokens. Labels are kept for the task-facing terms of the surrogate, such as cross entropy, but the calibration subset is not used to update model parameters during search.

1.4.2 Cached Teacher Signals

The teacher is fixed while candidate masks are being searched. Its outputs on the calibration subset are therefore computed once and reused. The cached signals are the teacher logits and the teacher hidden states at each layer.

The logits support the task-facing part of the metric. The hidden states support the residual-update distance, since layer updates are computed from consecutive hidden states. Caching these teacher signals removes the teacher forward pass from each candidate evaluation. Candidate scoring then requires forwarding only the masked student on the same calibration inputs and comparing its outputs to the cached teacher signals.

1.4.3 Cheap and Full Packs

The implementation uses two calibration fidelities. A cheap pack contains fewer calibration tokens and is used when many candidates must be screened quickly. A full pack contains more tokens and is used for periodic full evaluations, Hall-of-Fame candidates, and final reranking.

Both packs are built from the same calibration subset. The cheap pack gives the search enough signal to discard weak candidates at low cost. The full pack reduces noise when a candidate is important enough to affect the population or the final selected architecture.

For full evaluation, the implementation can average over multiple independently sampled full packs. This resampling is used before cohort ranks are computed, so the update-rank gate and the task-rank ordering depend on averaged component distances rather than on a single calibration draw.

1.5 The Genetic Search Algorithm

The search stage receives the fixed mask representation, the compute budget, the calibration packs, and the surrogate metric defined in the preceding sections. Its role is to select one feasible architecture for recovery. The search cannot use gradients with respect to the binary masks,

and exhaustive enumeration is infeasible. The procedure therefore uses an evolutionary search that keeps multiple feasible candidates alive, evaluates only a bounded number of masks, and returns the best mask found under the surrogate ranking.

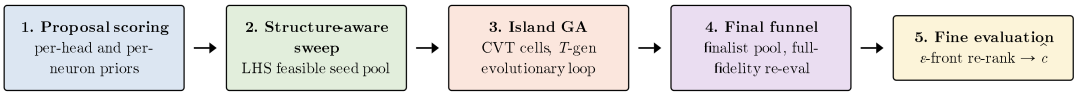


Figure 1.1: Five-phase overview of the genetic search. Phase 1 computes proposal scores from the teacher on the calibration set. Phase 2 builds a feasible seed pool by a structure-aware sweep. Phase 3 runs the island GA: a Centroidal Voronoi Tessellation (CVT) partition of $\mathcal{X}_B$ (Du et al., 1999) and a T-generation evolutionary loop with archive updates. Phase 4 pools finalists and re-evaluates them at full fidelity. Phase 5 selects $\widehat{c}$ from the pooled finalists.

1.5.1 Search Interface and Feasibility Objective

The previous sections define the pieces of the pruning problem in isolation. The search stage is where these pieces are assembled into a concrete selection process. Rather than changing the representation, the budget model, the calibration data, or the ranking metric, the search treats them as fixed inputs and uses them to identify one feasible architecture for recovery.

Figure 1.2 summarizes the interface of the search stage. The search receives four fixed objects from the previous sections: the candidate representation, the target compute budget, the calibration packs, and the surrogate metric. It returns a single mask $\hat{c}$ that satisfies the structural and computational constraints while minimizing the surrogate distance among the candidates explored by the search procedure.

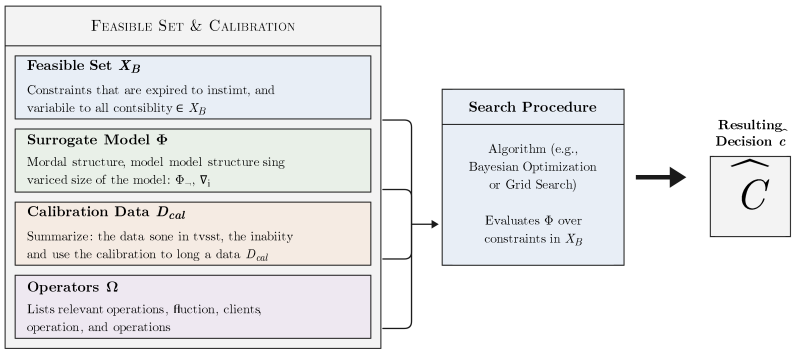


Figure 1.2: Inputs and output of the search stage. The candidate representation, compute budget, calibration packs, and surrogate metric are fixed before search. The search procedure explores feasible candidates and returns one selected mask $\hat{c}$ for recovery.

Feasibility of a candidate is determined by three conditions: its operation cost must lie within a tolerance band around the target budget, its mask must correspond to an executable forward pass, and at least $L_{\min}$ layers must remain active. The first condition is the binding constraint of the search and admits a compact formulation. A candidate

$$c = \left(M^H, \{n_l\}_{l=1}^{L}\right)$$

specifies a head mask and a per-layer neuron count, as defined in Section 1.1. Its operation cost $\mathrm{MAC}(c)$ follows from these choices through the cost model of Section 1.2. The cost model

restricts admissible candidates to the feasibility band

$$\mathcal{X}_B = \{c : (1-\varepsilon)B \le \mathrm{MAC}(c) \le (1+\varepsilon)B\}\,.$$

Inside this band, the surrogate distance Φ of Section 1.3 ranks candidates against the teacher on the calibration packs of Section 1.4. Two side conditions further restrict the feasible region: each mask must correspond to a realizable forward pass, and the active-depth floor

$$N_a(c) \ge L_{\min}$$

keeps at least $L_{\min}$ layers active. Equivalently, the complete feasible set can be written as

$$\mathcal{F}_B = \{c \in \mathcal{X}_B : c \text{ is executable } \wedge\ N_a(c) \ge L_{\min}\}\,.$$

The search returns the mask that minimises the surrogate distance over this restricted feasible set:

$$\hat{c} \in \arg\min_{c \in \mathcal{F}_B} \Phi(c).$$

The objective above defines the mask selected by our method. To approximate this objective under the finite evaluation budget, we organize the search as a structured genetic procedure over the feasible band $\mathcal{F}_B$. The procedure is designed to maintain diversity across distinct regions of the budget-constrained space, enforce feasibility after candidate modifications, and reserve higher-fidelity evaluation for the most competitive masks. The next subsection details this procedure and describes how it produces the final selected mask $\hat{c}$.

1.5.2 The Five-Phase Genetic Procedure

The combinatorial size of $\mathcal{X}_B$ and the bounded evaluation budget shape the five-phase structure of the procedure. Phases 1 and 2 prepare the search state with per-head and per-neuron proposal scores and a feasible seed pool. Phase 3 runs the iterated evolutionary loop over regional populations, accumulating best candidates into three global archives. Phases 4 and 5 collapse the search state: a final funnel re-evaluates pooled finalists at full fidelity, and a fine-evaluation step selects $\widehat{c}$ under an ϵ-front tie-break.

Phase 1: Proposal Scoring

Before any candidate is constructed, the procedure computes per-head and per-neuron Fisher-style local sensitivities of the teacher on the calibration set, denoted $s_H[\ell, h]$ and $s_N[\ell, n]$. The per-layer aggregate

$$\mathrm{imp}(\ell) \;=\; \sum_h s_H[\ell, h] \;+\; \sum_n s_N[\ell, n]$$

serves as a layer importance prior. These quantities are not the search objective; the surrogate Φ of Section 1.3 remains the objective. The proposal scores serve three roles: they decide which heads and neurons each operator adds or removes, they order block-level additions and removals during repair, and they define the ranking of layers from which the protected core during repair is drawn.

Phase 2: Structure-Aware Sweep

The initial seed pool is built by a Latin hypercube sweep (McKay et al., 1979) over three orthogonal axes: the active-layer count N_a, the fraction ϕ_H of the operation budget spent on attention heads, and a subset mode that decides how the active layers are drawn from $\mathrm{imp}(\ell)$. For each cell of the LHS grid, a candidate is constructed with the chosen N_a active layers and the chosen split of MAC between heads and neurons, then repaired into $\mathcal{X}_B$. The sweep returns

a pool of feasible candidates spread across coarse macro descriptors. This pool serves two roles in Phase 3: it seeds the per-region populations, and it provides the sample of architectures from which $\mathcal{X}_B$ is partitioned into regions of similar candidates by a *Centroidal Voronoi Tessellation* (CVT) (Du et al., 1999), a k-means-style clustering procedure that groups masks of similar architectural shape into a small number of regions, each represented by the average of its members. The construction is given in Phase 3.

Phase 3: Island GA

The third phase searches the feasible band $\mathcal{X}_B$ through multiple populations, with each population anchored in a different region of the band. This allows local refinement and broader exploration to proceed at the same time. Some feasible masks are close neighbours: they differ only by small budget-preserving edits, such as swapping active heads, moving neuron blocks within or across layers, or adjusting the head–neuron balance while keeping the MAC cost nearly fixed. These edits preserve the candidate's overall structure, so they are well suited to local refinement.

Other masks can satisfy the same MAC budget while allocating capacity in structurally different ways. For example, one mask may keep most capacity in early layers, another may shift it toward later layers, and another may change which layers remain active at all. These masks occupy distinct regions of $\mathcal{X}_B$, not merely small perturbations of the same candidate. The search therefore treats them as exploration targets rather than as local refinements.

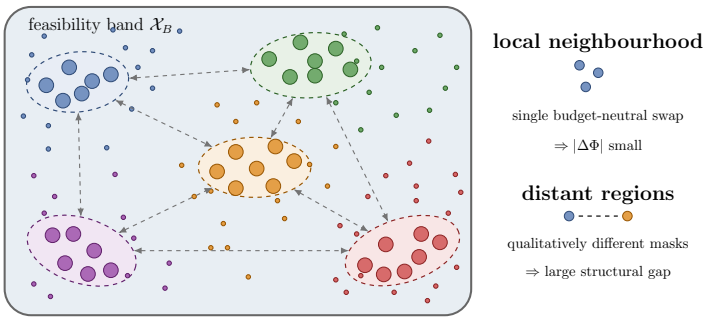


Figure 1.3: The feasibility band $\mathcal{X}_B$ as a populated space. Feasible candidates form clusters of architecturally similar masks; clusters in different regions of the band differ qualitatively.

The third phase encodes this two-scale structure through a *Centroidal Voronoi Tessellation* (CVT) of $\mathcal{X}_B$ (Du et al., 1999), constructed by k-means on a six-coordinate architectural descriptor $z(c) \in [0,1]^6$ fitted to the sweep pool. The coordinates capture the share of MAC spent on heads, the neuron fraction, the active-layer fraction, the entropy of the per-layer budget distribution, and the centres of mass of heads and of neurons along the depth axis. Each Voronoi cell defines an island's basin; a population of size P is seeded inside each basin from the sweep pool of Phase 1.5.2, projected into the basin's bounds. Local refinement is then carried out inside each island, while the partition itself enforces coverage across regions.

Within this partition, the loop runs for T generations (Figure 1.5). In each generation, per island: parents are chosen by a tournament rule whose score combines a fitness-shared rank (each rank divided by a structural-crowding count) with a novelty bonus equal to the minimum mask-distance to already-selected parents; offspring are produced by crossover and mutation operators drawn from an EXP3-IX bandit local to the island; each offspring is projected back into the basin to enforce the budget band and the basin bounds; offspring are evaluated by Φ at cheap fidelity, with a scheduled subset re-evaluated at full fidelity; and the union of incumbents and offspring is ranked by the two-step cohort rule of Section 1.3.5. Survivors form the next-generation population. The best-scoring candidates of the cohort are inserted into three global archives: a Hall of Fame keyed by exact mask signature, a Pareto front on (Φ, MAC),

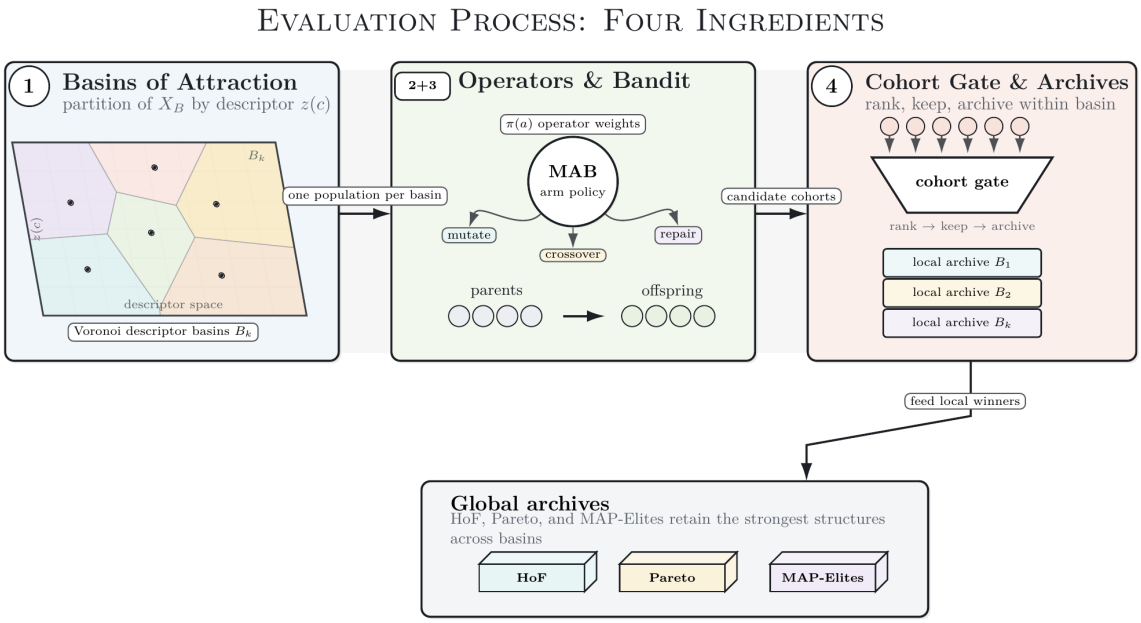


Figure 1.4: The four pieces of machinery inside the generational loop. Regions maintain separate populations, operators produce feasible offspring after repair, bandits adapt operator probabilities, and selection feeds both the next population and the archives.

and a MAP-Elites grid keyed by (N_a, ϕ_H).

Phase 4: Final Funnel

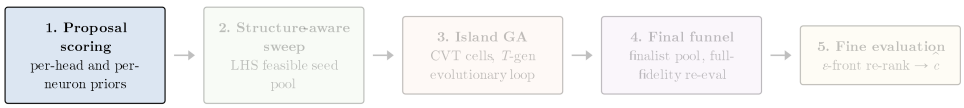
After T generations, the procedure assembles a finalist pool from four sources: the global best candidate observed across the run, the top candidates of each island's final population, samples from the three archives (Hall of Fame, Pareto front, MAP-Elites), and the global best running incumbent (Figure 1.5). The pool is deduplicated by exact mask signature. Each finalist is then re-evaluated on the full-fidelity calibration pack with R independent resamples averaged into a single score.

Phase 5: Fine Evaluation

The averaged finalist scores define an ϵ-front: the candidates whose resampled Φ lies within ϵ of the lowest score in the finalist pool. The ϵ-front is sorted lexicographically by $(\Phi, -\text{MAC})$, so among candidates within calibration noise of one another the procedure prefers the one that uses more of the allowed compute. The first element of the sort is returned as $\widehat{c}$ (Figure 1.5).

1.6 The Detailed Pipeline

1.6.1 Proposal Scoring



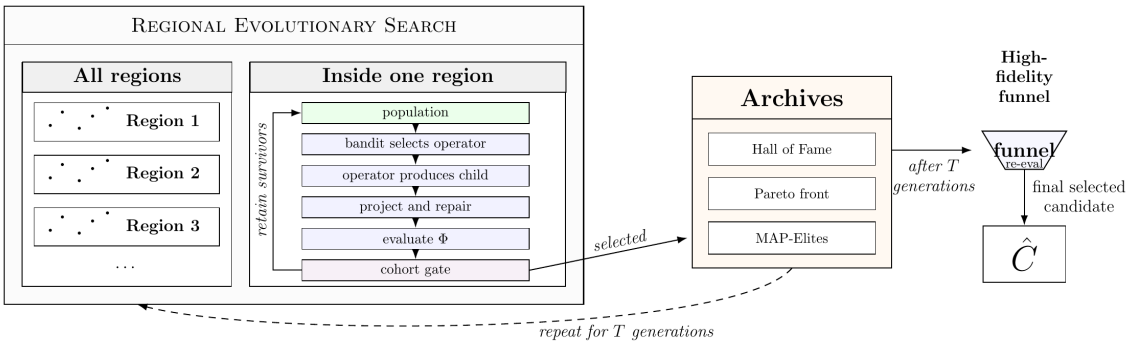


Figure 1.5: One iteration of the search and the final funnel. Each region's population produces offspring with operators chosen by its bandit; offspring are projected back into the region, evaluated on the calibration packs, ranked by the cohort gate, and used to update the population and the archives. After T iterations, the archives and the per-region best candidates are re-evaluated at higher fidelity and one mask $\widehat{c}$ is returned.

The first phase computes proposal scores before any candidate mask is sampled. These scores are not the search objective; they are local saliency estimates used to make later discrete edits less arbitrary. The intuition follows classical pruning saliency: a unit is valuable when a small change to its gate would strongly affect the objective, or when removing it directly increases reconstruction error (**???**Michel et al., 2019). Here the saliency is computed for heads and feed-forward neurons against the teacher's per-layer residual update target on the calibration set of Section 1.4.

For head h of layer ℓ, let $g_{\ell,h}(x)$ denote the gradient of the teacher-forced per-layer update target with respect to a multiplicative gate placed at head (ℓ, h) for calibration example x. This gradient measures local sensitivity: if the squared gradient is large, changing the gate of that head is expected to disturb the layer update. The head proposal score combines this local sensitivity with an explicit zero-out reconstruction term:

$$s_H[\ell, h] \;=\; \mathbb{E}_{x\sim\mathcal{D}_{\text{cal}}}\big[\, g_{\ell,h}(x)^2 \,\big] + \lambda \cdot \mathrm{MSE}\Big(\widetilde{\Delta}_\ell^{(\mathcal{T},\,h=0)}, \Delta_\ell^{(\mathcal{T})}\Big)\,,$$

where λ is a fixed mixing coefficient. The MSE term measures how much the teacher's per-layer update changes when head (ℓ, h) is zeroed. The neuron score $s_N[\ell, n]$ is defined analogously by placing the gate on feed-forward neuron (ℓ, n). Both scores are row-normalised per layer, so they rank units within the same layer rather than comparing raw magnitudes across layers.

The search also needs a layer-level score because the sweep and the repair routine sometimes decide which layers should remain active before they decide which exact units to keep. The per-layer aggregate

$$\mathrm{imp}(\ell) \;=\; \sum_h s_H[\ell, h] \;+\; \sum_n s_N[\ell, n]$$

gives this single importance number per layer. Phase 2 uses it when choosing active layers, and repair uses it to define the protected core.

Neuron mutation and repair operate on blocks rather than individual neurons. To make block addition and removal efficient, each layer receives a cumulative block table. For layer ℓ, let π_ℓ be the permutation that sorts $s_N[\ell, \cdot]$ in descending order. The cumulative score of the first b neuron blocks is

$$S_\ell[b] \;=\; \sum_{j=1}^{b\cdot b_n} s_N[\ell,\, \pi_\ell(j)], \qquad b = 0, 1, \ldots, \lfloor N/b_n \rfloor.$$

The marginal gain of adding the next block to layer ℓ is $S_\ell[b{+}1] - S_\ell[b]$, and the marginal loss of removing the current last block is $S_\ell[b] - S_\ell[b{-}1]$. After the one-time descending sort, both

queries are constant-time. This matters during mutation and repair because many candidate edits require quick decisions about where to add or remove neuron capacity.

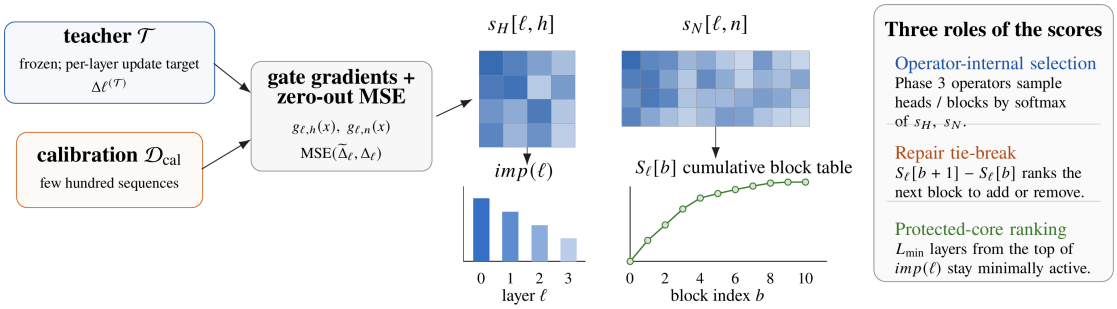


Figure 1.6: Proposal scoring. For each calibration batch, gradients of the teacher's per-layer update target with respect to head and neuron gates yield squared sensitivities and a reconstruction MSE; their combination, row-normalised per layer, produces $s_H[\ell, h]$ and $s_N[\ell, n]$. Summing over each layer gives $\text{imp}(\ell)$. Sorting $s_N[\ell, \cdot]$ in descending order and accumulating block sums yields the table $S_\ell[b]$, from which marginal gain and loss queries take $O(1)$.

The scores then flow into the rest of the search in three places. First, the mutation operators of Phase 3 use s_H and s_N to define sampling probabilities when heads or neuron blocks are added, removed, or swapped. Second, the repair routine uses S_ℓ to choose the best block to add or the least damaging block to remove when a candidate must be projected back into the feasible band. Third, the guarded core during repair is formed from the L_{min} most important layers, ensuring that repair does not accidentally deactivate every high-priority layer while satisfying the budget.

1.6.2 Initialisation: Structure-Aware Sweep

The first phase provides proposal scores for heads, neurons, and layers. These scores indicate which structures appear locally promising, but they do not yet form a diverse population of feasible architectures. The next step is therefore to construct an initial pool of candidates inside $\mathcal{X}_B$, the set of masks whose operation cost lies within the target tolerance band. This pool must be diverse because it becomes the material from which the island construction of Section 1.6.3 defines and seeds separate search regions. To obtain this coverage, the sweep uses a stratified Latin hypercube design (McKay et al., 1979) over three macro axes. Each sampled design point produces one candidate, so the initial pool covers active depth, head–neuron budget balance, and active-layer selection in a controlled way.

The first axis is the active-layer count N_a, ranging over $\{L_{\text{min}}, \ldots, L_{\text{max}}\}$. The lower bound L_{min} is the active-depth floor of the search. The upper bound is the largest number of minimally complete active layers that can fit inside the budget:

$$L_{\text{max}} = \min\left(L, \left\lfloor \frac{B}{c_H + b_n c_N} \right\rfloor\right).$$

Here $c_H + b_n c_N$ is the minimum cost of keeping one layer active with one attention head and one block of feed-forward neurons. The second axis is the head-MAC fraction $\phi_H \in \{0.05, 0.10, 0.15, 0.25, 0.35, 0.50, 0.65, 0.80, 0.92\}$, which controls how the budget is split between attention and feed-forward capacity and ranges from neuron-dominated to attention-dominated configurations. The third axis is a subset mode that selects which N_a layers are kept active given the importance vector $\mathrm{imp}(\ell)$, drawn from $\{\texttt{topk}, \texttt{noisy\_topk}, \texttt{weighted}, \texttt{random}\}$. The subset mode may optionally be paired with a partial-layer style that injects single-branch layers (attention-only or feed-forward-only) into the sweep pool.

Once N_a is fixed, the sweep still has to decide which layers are active. This decision is controlled by the subset mode. The most prior-driven choice, `topk`, concentrates the active set on the layers with the largest importance scores. The `noisy_topk` choice keeps the same preference for strong layers after perturbing the scores, which reduces repeated selection of the same active-layer pattern. The `weighted` choice samples layers with probabilities derived from their importance scores, allowing plausible weaker layers to enter the pool. The `random` choice samples active layers uniformly and supplies a prior-free source of architectural diversity.

Each sampled design point is therefore a template for one candidate: it fixes how many layers are active, how much of the budget is assigned to attention, and how the active layers are chosen. The construction turns this template into a mask in three steps.

First, the subset mode selects the active layer set $A \subset \{1, \ldots, L\}$, where A contains the layers that will receive nonzero heads or neurons in the candidate. Second, the budget is distributed across these selected layers. Let $\mathrm{importance}(\ell)$ denote the layer-level proposal score from Phase 1, and let η_ℓ be a positive random perturbation for layer ℓ. The unnormalised layer weight w_ℓ combines the importance score with this perturbation, so stronger layers receive larger shares while repeated samples with similar priors can still produce different allocations:

$$w_\ell = \mathrm{importance}(\ell)\eta_\ell, \qquad \tilde{w}_\ell = w_\ell \,/\, \textstyle\sum_{k \in A} w_k, \qquad \ell \in A.$$

The normalised weight $\tilde{w}_\ell$ is therefore the fraction of the candidate budget assigned to active layer ℓ.

Third, the head fraction ϕ_H divides each layer's assigned budget into attention and feed-forward capacity. With target budget B, per-head cost c_H, per-neuron cost c_N, and neuron block size b_n, the continuous layer budgets are converted into integer head counts and block-aligned neuron counts:

$$a_\ell = \mathrm{round}\left(\frac{\phi_H B \tilde{w}_\ell}{c_H}\right), \qquad n_\ell = b_n \left\lfloor \frac{(1-\phi_H) B \tilde{w}_\ell}{b_n c_N} \right\rfloor.$$

The counts are clipped to valid layer ranges. The active head identities are then sampled from the head proposal scores $s_H[\ell, \cdot]$, so high-scoring heads are more likely to be retained. Layers outside A receive zero heads and zero neurons.

Rounding and block alignment can move the final operation count slightly outside the feasibility band. Budget repair projects the candidate back into $\mathcal{X}_B$ by adding or removing block-aligned structures according to the proposal scores until the cost satisfies the band. Applying this construction to all Latin hypercube samples produces the feasible seed pool used by the regional populations and by the partition fit of the next phase (Du et al., 1999).

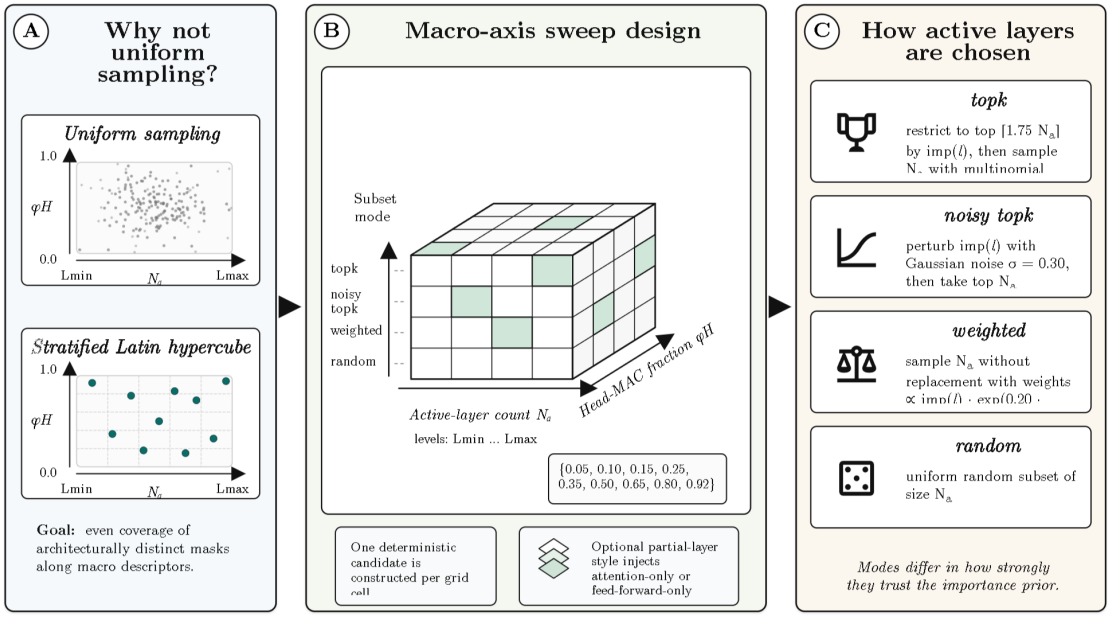
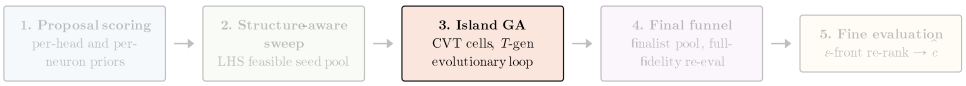


Figure 1.7: Structure-aware sweep. Each sampled design point $(N_a, \phi_H, \text{mode})$ yields one candidate. The subset mode chooses the active layers from the layer importance scores; the head–neuron split ϕ_H divides the budget; and the importance-weighted per-layer allocation distributes the split into integer head counts and block-aligned neuron counts.

1.6.3 CVT Partition and Island Construction



The sweep of Section 1.6.2 produces a scored feasible pool, denoted

$$\mathcal{S} = \{x_i\}_{i=1}^{M}.$$

Each x_i is a candidate mask that satisfies the budget band, and its score $\Phi(x_i)$ is the surrogate distance defined in Section 1.3. Lower values indicate closer agreement with the teacher on the calibration packs, so Φ orders candidates by quality.

Island construction requires a second view of the same pool. The strongest candidates may still belong to different allocation regimes: some spend most remaining operations on attention, some preserve more feed-forward width, and some concentrate the budget into fewer active layers. These regimes should not be mixed before local evolution begins, because mutation operators act differently in each regime. A head swap, a neuron transfer, or a layer edit has a different effect depending on the allocation pattern of the parent mask.

Phase 3 therefore groups candidates by architectural shape and attaches one regional population to each group. This follows the island model of evolutionary search, where each island maintains a subpopulation inside one part of the search space (Cantú-Paz, 2000). In this procedure, an island also keeps its own incumbent candidate and operator-selection statistics. The clustering step below defines the regions through a descriptor $z(x_i)$, then assigns one island to each region.

Architectural Descriptor. Clustering requires a numerical representation of architecture shape. Each candidate x is therefore mapped to a six-dimensional descriptor

$$z(x) = (z_1(x), \ldots, z_6(x)) \in [0,1]^6.$$

The coordinates capture the allocation properties that determine whether two masks should be refined by the same local mutation process. Table 1.2 gives the role of each coordinate, and the equations that follow define how these quantities are computed from the mask.

Coordinate	Quantity	Interpretation
z_1	attention MAC share	separates head-heavy from neuron-heavy allocations
z_2	surviving feed-forward width	measures how much dense feed-forward capacity remains
z_3	complete active-layer fraction	measures how much depth retains both branches
z_4	budget entropy across layers	distinguishes spread-out and concentrated allocations
z_5	head centre of mass	locates attention capacity along depth
z_6	neuron centre of mass	locates feed-forward capacity along depth

Table 1.2: Six descriptor coordinates used to partition the feasible sweep pool.

The descriptor begins with the two global counts induced by the mask. For candidate x, $a_\ell(x)$ is the number of active attention heads in layer ℓ, and $n_\ell(x)$ is the number of active feed-forward neurons in that layer. Summing these counts over all layers gives the total surviving attention and feed-forward capacity:

$$H_{\mathrm{tot}}(x) = \sum_\ell a_\ell(x), \qquad N_{\mathrm{tot}}(x) = \sum_\ell n_\ell(x).$$

These totals define the first view of the candidate: its balance between attention and feed-forward capacity. The constants c_H and c_N are the operation costs of one active head and one active feed-forward neuron from the budget model. The coordinate z_1 is the fraction of the mask-dependent cost assigned to attention, and z_2 is the surviving feed-forward width normalised by the dense feed-forward width LN:

$$z_1(x) = \frac{H_{\mathrm{tot}}(x)c_H}{H_{\mathrm{tot}}(x)c_H + N_{\mathrm{tot}}(x)c_N}, \qquad z_2(x) = \frac{N_{\mathrm{tot}}(x)}{LN}.$$

The next view describes how the surviving computation is distributed across depth. The coordinate z_3 measures how many layers remain complete, meaning that both the attention branch and the feed-forward branch are still active:

$$z_3(x) = \frac{|\{\ell : a_\ell(x) > 0 \wedge n_\ell(x) > 0\}|}{L},$$

The coordinate z_4 measures whether the candidate spreads its budget over many layers or concentrates it in a small number of layers. For this calculation, b_ℓ is the operation cost assigned to layer ℓ, and p_ℓ is the fraction of the candidate's layer-wise budget carried by that layer:

$$z_4(x) = -\frac{1}{\log L} \sum_{\ell : b_\ell > 0} p_\ell \log p_\ell, \qquad p_\ell = \frac{b_\ell}{\sum_k b_k}, \qquad b_\ell = a_\ell(x)c_H + n_\ell(x)c_N.$$

The entropy coordinate z_4 is large when the budget is distributed across depth and small when the budget is concentrated. The final view records where the surviving capacity is located. The layer index ℓ acts as a depth coordinate; division by $L-1$ normalises the weighted average into the unit interval. Thus z_5 is the centre of mass of active heads, and z_6 is the centre of mass of active feed-forward neurons:

$$z_5(x) = \frac{1}{(L-1)H_{\mathrm{tot}}(x)} \sum_\ell \ell\, a_\ell(x), \qquad z_6(x) = \frac{1}{(L-1)N_{\mathrm{tot}}(x)} \sum_\ell \ell\, n_\ell(x).$$

Low values of z_5 or z_6 indicate capacity placed near the lower layers, and high values indicate capacity placed near the upper layers. Taken together, the six coordinates place masks with similar allocation balance and similar depth distribution close to one another. This gives the clustering step a shape representation, while $\Phi(x)$ remains the ranking signal for candidate quality.

This descriptor thus projects each possible mask to a point in this six dimensional shape space described above. However, such a point still represents positions relative to other candidates rather than actual search region parameters. This is why the set of descriptor clouds is then clustered into a small set of regions where similarity among points is closer in terms of structural proximity, so simply in terms of allocation, active depth, budget, spread, and capacity at each candidate.

Descriptor Standardisation and CVT Fitting. The descriptor now gives a common language for architecture shape. Before distance-based clustering is applied, the descriptor coordinates are standardised because their empirical spreads can differ across the sweep pool:

$$\tilde{z}_i = \frac{z(x_i) - \bar{z}}{\sigma_z},$$

Here $z(x_i)$ is the descriptor of sweep candidate x_i, $\bar{z}$ is the componentwise sample mean, σ_z is the componentwise sample standard deviation, and $\tilde{z}_i$ is the standardised descriptor used for clustering. Standardisation places all six coordinates on a common numerical scale.

The standardised descriptors are then partitioned by a *Centroidal Voronoi Tessellation* (CVT) (Du et al., 1999). The fitting procedure uses Lloyd's alternating assignment and centroid-update steps, as in k-means clustering (Lloyd, 1982; MacQueen, 1967). A detailed background on Lloyd iteration and Voronoi regions is reserved for Appendix **??**.

Five centres are initialised in standardised space by farthest-point seeding. The first centre $\tilde{\mu}_1$ is drawn uniformly from $\{\tilde{z}_i\}$, and each subsequent centre is selected from the least covered part of the descriptor cloud:

$$\tilde{\mu}_{j+1} = \arg\max_{\tilde{z}_i} \min_{1 \le q \le j} \|\tilde{z}_i - \tilde{\mu}_q\|_2^2.$$

The Lloyd assignment step maps each descriptor to its closest centre,

$$\text{label}(i) = \arg\min_{1 \le j \le k} \|\tilde{z}_i - \tilde{\mu}_j\|_2^2,$$

and the update step replaces each centre by the mean descriptor assigned to it:

$$\tilde{\mu}_j \leftarrow \frac{1}{|C_j|} \sum_{i \in C_j} \tilde{z}_i, \qquad C_j = \{i : \text{label}(i) = j\}.$$

After convergence, the centres are mapped back to the original descriptor coordinates by

$$\mu_j = \sigma_z \tilde{\mu}_j + \bar{z}.$$

The resulting cells are nearest-centre regions in descriptor space:

$$V_j = \left\{ z \in [0,1]^6 : \|z - \mu_j\|_2 \le \|z - \mu_q\|_2 \text{ for all } q \ne j \right\}.$$

Each cell groups candidates with similar head–neuron balance, active depth, budget spread, and depth centre of mass. These cells provide the regions from which island populations are constructed.

Island Labels. After the cells are fitted, their centroids provide interpretable names for the regions. A centroid with high attention share and low feed-forward width receives a head-heavy label. A centroid with high feed-forward width receives a neuron-heavy label. A centroid with low active-depth receives a layer-sparse label. Intermediate centroids are labelled balanced or head-leaning according to their head–neuron balance. These labels are descriptive names for the discovered regions; the optimisation uses the numeric basin constraints defined below.

The labelled cells define five regional hypotheses under the same MAC budget. Head-favouring islands test allocations where attention receives a large part of the remaining computation. The balanced island keeps the search near the middle of the head–neuron trade-off. The neuron-heavy island tests the opposite allocation pattern. The layer-sparse island focuses on masks that retain fewer active layers and assign more capacity to those layers. Maintaining these islands keeps the search from collapsing all allocation regimes into one shared population.

From Cells to Basin Constraints. The labels identify the search regimes, and the next step makes those regimes enforceable. The CVT cells live in descriptor space, while mutation operators act on masks. Each cell is therefore converted into constraints that can be checked after an offspring is created. For cell j, the sweep candidates assigned to V_j are summarised by empirical bounds on their structural counts, attention share, utilisation, cost floor, centroid, and descriptor radius. This gives the basin

$$\mathcal{B}_j = \big(h^j_{\min}, h^j_{\max}, n^j_{\min}, n^j_{\max}, [\phi^j_-, \phi^j_+], [u^j_-, u^j_+], \rho^j_{\min}, \mu_j, r_j\big).$$

The head bounds $(h^j_{\min}, h^j_{\max})$ are empirical percentiles of H_{tot} inside the cell, and $(n^j_{\min}, n^j_{\max})$ are the corresponding block-aligned percentiles of N_{tot}. The interval $[\phi^j_-, \phi^j_+]$ bounds the attention-cost share z_1. The utilisation interval $[u^j_-, u^j_+]$ bounds the per-layer cost intensity, with layer utilisation computed from the layer cost b_ℓ relative to the dense per-layer cost. The scalar $\rho^j_{\min}$ is a lower bound on $\text{MAC}(x)/B$, which prevents under-budget candidates from entering the island. The centroid μ_j and radius r_j enforce proximity to the cell in descriptor space.

Projection uses these basin constraints after mutation. It adjusts structural counts to satisfy the basin ranges, applies the budget repair from the sweep construction, and keeps the resulting mask inside the feasibility band. This projection step connects the geometric partition to executable offspring.

Island Seeding. Once each cell has an enforceable basin, the score Φ re-enters the construction. Candidates assigned to cell j are ordered by their surrogate distance, and the strongest feasible candidates seed the island attached to $\mathcal{B}_j$. If a cell has too few assigned candidates, nearby sweep candidates are projected into the basin before insertion. Every seed is therefore feasible before the genetic loop begins.

The five basins define the five islands $\mathcal{I}_1, \ldots, \mathcal{I}_5$. Island $\mathcal{I}_j$ is the running state attached to basin $\mathcal{B}_j$:

$$\mathcal{I}_j = \big(\mathcal{P}_j, x^\star_j, d^\star_j, \mathcal{A}_j\big).$$

Here $\mathcal{P}_j$ is a population of P candidates satisfying $\mathcal{B}_j$, and $(x^\star_j, d^\star_j)$ stores the best mask observed in basin j and its surrogate distance. The bandit state $\mathcal{A}_j$ is the operator-selection mechanism used later in the generation loop; it maintains weights over edit operators and updates them from observed rewards, following the multi-armed bandit setting (Auer et al., 2002; Neu, 2015). Phase 3 then starts from five feasible regional populations, each tied to one allocation hypothesis and ready for local evolutionary refinement.

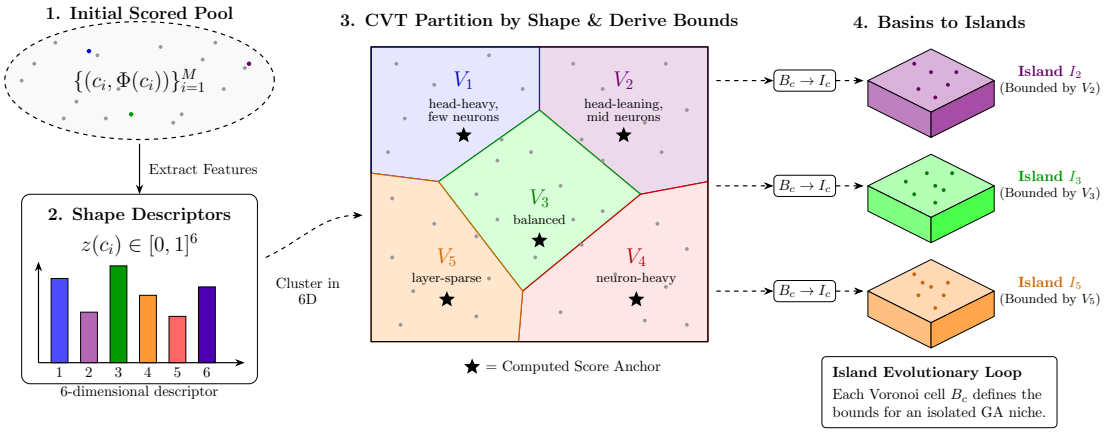
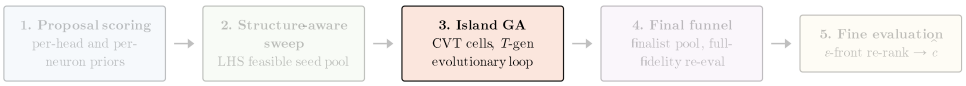


Figure 1.8: From scored sweep candidates to islands. Sweep candidates are projected into the (z_1, z_3) plane on the left and form a continuous cloud. The centre-based clustering step assigns each descriptor to its nearest centre and updates each centre to the mean of its assigned descriptors. This produces five Voronoi cells. Each cell becomes a basin $\mathcal{B}_j$ on the right; the best scored candidates assigned to that basin seed the island $\mathcal{I}_j$, and the island then evolves with its own population, best-so-far candidate, and operator-selection bandit.

1.6.4 Operator Library



With basins and populations in place, each island needs controlled variation. The search uses two scales of mutation. The first scale performs local edits that refine a candidate inside its current basin. These edits preserve the broad architectural shape and mainly exchange units of similar cost. The second scale performs structural edits that alter the allocation pattern, such as changing active depth or moving compute between branches. This two-scale design allows an island to improve a local mask while still testing larger architectural changes when local edits stop producing useful offspring.

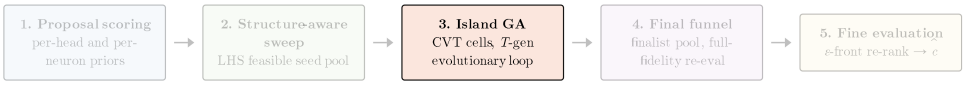
A micro mutation is a local replacement. For example, a budget-neutral head swap removes one active head and activates another head of the same cost, guided by the proposal scores s_H. The total head count remains fixed, and the candidate stays close to its current basin. A macro mutation changes a larger structural decision. For example, a layer-rewiring mutation changes which layers are active while preserving the required active-depth and budget constraints after projection. The full inventory of mutation operators is placed in a separate appendix-ready table.

Crossover is distinct from mutation. Mutation edits one parent, while crossover combines two parents into one offspring. Its purpose is to recombine useful layer allocations that were discovered separately. One parent may contribute the active-layer topology, another may contribute branch widths or layer-wise counts, and the resulting child is then projected back into the basin. The projection step is required because recombination can produce a valid-looking mask whose cost or basin descriptor has moved outside the allowed range.

During offspring generation, the algorithm chooses between mutation and crossover according to the current operator mode. When mutation is selected, the bandit of Section 1.6.5 chooses an

operator from the relevant family. When crossover is selected, two parents are combined and the child is repaired through the same projection mechanism. Thus the operator library defines the local and structural edits, while crossover supplies recombination across already discovered candidates.

1.6.5 Bandit and Niche Conditioning



Once the operator families are defined, the search must decide which move to apply for each offspring. This is a local decision because the useful operator can differ from one island to another. A layer-sparse island benefits from edits that change active depth, while a head-favouring island benefits from edits that adjust the head–neuron balance. The procedure models this choice as an operator-selection problem and assigns one multi-armed bandit to each island (Auer et al., 2002; Neu, 2015).

For island i, the maintained object is a positive weight vector $w_i[o]$ over the available operators. A larger weight means that operator o has produced useful offspring more often in that island. These weights define the base sampling distribution

$$p_i[o] = (1-\gamma)\frac{w_i[o]}{\sum_{o'} w_i[o']} + \frac{\gamma}{K},$$

where K is the number of available operators and γ is the exploration coefficient. The first term favours operators with larger accumulated weight. The uniform term assigns every operator a nonzero probability, so rarely used operators can still be tested.

After an operator is sampled and its offspring is evaluated, the observed reward is assigned only to the sampled operator. The update therefore uses an importance-style reward estimate. If $\hat{p}$ is the probability used to sample operator o, the implicit-exploration estimate is

$$\hat{r}_o = \frac{r_o}{\hat{p} + \gamma}.$$

The island then applies the multiplicative update

$$w_i[o] \leftarrow w_i[o] \exp\left(\frac{\gamma \hat{r}_o}{K}\right).$$

This is the EXP3-IX update. The additional γ in the denominator controls the size of reward estimates for low-probability operators, keeping the update stable when an operator has been sampled rarely. Weights are periodically rescaled for numerical stability.

The initial weights encode the basin focus obtained in Section 1.6.3. A layer-sparse basin starts with larger weights on depth-changing operators. A head-heavy basin starts with larger weights on operators that move capacity toward attention. Balanced and neuron-heavy basins receive analogous initial preferences. These priors only initialise the search; the multiplicative updates can change the distribution once observed rewards contradict the initial focus.

The island-level bandit gives one distribution for the whole island. Parents inside the same island can still differ in local structure, so the procedure adds a parent-specific correction through niches. A niche is a coarse descriptor of the parent,

$$\nu(c) = (N_a,\ \text{mac\_bucket},\ \text{shape\_bucket}).$$

The first component records active depth. The second component discretises the candidate's relative cost $\mathrm{MAC}(c)/B$. The third component records whether the parent is head-heavy, balanced, or neuron-heavy according to its head-versus-neuron utilisation. For each pair (ν, o), the island maintains an exponentially weighted moving average of the reward obtained when operator o was used on parents of niche ν:

$$\mathrm{ema}_i[\nu, o] \leftarrow (1 - \eta_{\mathrm{ema}})\mathrm{ema}_i[\nu, o] + \eta_{\mathrm{ema}} r_o.$$

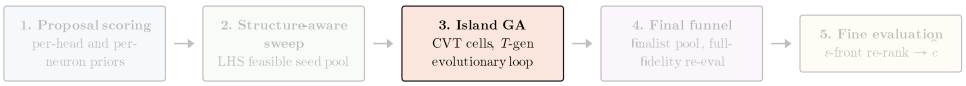
This niche statistic modifies sampling without replacing the island-level bandit. When a parent has niche ν, the base probability $p_i[o]$ is reweighted by the corresponding niche average and renormalised:

$$\tilde{p}_i[o; \nu] \propto p_i[o] \left(0.5 + \mathrm{ema}_i[\nu, o]\right), \qquad \mathrm{ema}_i[\nu, o] \in [0, 1].$$

The resulting distribution $\tilde{p}_i[\cdot; \nu]$ is the distribution used to sample the operator for that parent. It combines island-level evidence with parent-specific evidence.

The reward r_o is defined after the offspring is compared with its parent by the two-step cohort rule of Section 1.3.5. The reward is positive when the offspring improves on its parent under that rule. A small additional bonus is assigned when the offspring also improves the island's best-so-far candidate. Thus the bandit receives credit both for local parent improvement and for progress at the island level.

1.6.6 The Generation Loop



With the islands initialised and the operator distributions in place, the search enters the main generational loop of Phase 3. One generation updates every island once. The loop has a fixed order: score the current population, select parents, preserve elites, generate offspring, project them into the basin, evaluate them, apply the cohort rule, update the population, and pass strong candidates to the archives of Section 1.6.7.

Population Scoring. At the start of generation t, island i holds a population

$$\mathcal{P}_i^{(t)} = \{c_{i,1}, \ldots, c_{i,P}\},$$

all of whose members satisfy the basin constraints of Section 1.6.3. The loop first evaluates every candidate at cheap fidelity on the calibration packs, producing a surrogate distance $\Phi(c_{i,j})$. Since lower distance is better, the distance is converted into a maximisation-oriented raw fitness

$$f_{i,j} = -\Phi(c_{i,j}).$$

This raw fitness ranks candidates by quality inside the current island. Before parent selection, it is corrected for crowding so that a dense group of near-identical masks does not dominate the parent pool.

The correction is a standard fitness-sharing rule. The distance D_{jk} is the normalised Hamming distance between the exact binary mask signatures of candidates $c_{i,j}$ and $c_{i,k}$ in the same island, and σ_s is the sharing radius. The shared fitness of candidate j is

$$\tilde{f}_{i,j} = \frac{f_{i,j}}{1 + \sum_{k \neq j} \max\left(0, 1 - \frac{D_{jk}}{\sigma_s}\right)}.$$

Candidates surrounded by many near-duplicates receive lower selection strength. More isolated candidates retain more of their raw fitness. This keeps the island from collapsing too quickly onto one local shape.

Parent Selection and Elites. Parent selection is then applied on top of the shared fitness. The candidate pool is formed from the highest shared-fitness members of the island. Parents are drawn by repeated tournament selection. If the algorithm has already chosen parents $p_1, \ldots, p_{k-1}$ for the current offspring batch, then a tournament candidate $c_{i,j}$ receives the score

$$S_{i,j}^{\text{parent}} \;=\; \tilde{f}_{i,j} \;+\; \alpha_{\text{nov}} \min_{k'<k} D_{j,p_{k'}} \;+\; \alpha_{\text{tie}} u, \qquad u \sim \text{Unif}[0,1].$$

The second term is a novelty bonus weighted by α_{nov}; among candidates with similar shared fitness, it favours candidates far from the parents already chosen for the same batch. The final term is a small random tie-break weighted by α_{tie}, used only to avoid deterministic cycles.

Before any offspring are created, the island preserves a small elite set unchanged. If the global elite budget is elite_size and the number of islands is I, then island i copies forward

$$e = \left\lceil \frac{\text{elite\_size}}{I} \right\rceil$$

top candidates from $\mathcal{P}_i^{(t)}$ directly into the next population. The remaining slots are filled with offspring. The fraction of those offspring allocated to macro operators is controlled by a schedule $\beta_M(t)$. Early generations emphasise broader structural edits. Later generations shift probability mass toward micro operators once each island has established its local region.

Offspring Generation and Evaluation. Each remaining population slot is filled independently. The bandit of Section 1.6.5 samples an operator with the niche-conditioned distribution $\tilde{p}_i[o;\nu]$ of the selected parent. A crossover-or-mutation decision then determines whether the child is produced from one parent by the sampled mutation operator or from two parents by crossover. In both cases, the child is projected back into the basin so that it satisfies the budget band and the island bounds.

Before scoring, the offspring batch is filtered for repeated masks. Exact duplicates are removed by exact mask signature. A second pass removes structural near-duplicates using the same normalised mask distance and a deduplication radius σ_d. This prevents the offspring cohort from spending evaluations on masks that differ only by negligible local perturbations.

Every surviving offspring is evaluated at cheap fidelity. At scheduled generations, the strongest cheap-fidelity offspring are re-evaluated at full fidelity. If $\mathcal{D}_{\text{cal}}^{\text{full},r}$ denotes the r-th full calibration resample and R the resample count, then the full-fidelity score used in the loop is

$$\Phi^{\text{full}}(c) \;=\; \frac{1}{R} \sum_{r=1}^{R} \Phi\left(c; \mathcal{D}_{\text{cal}}^{\text{full},r}\right).$$

The cheap score remains the working ranking signal inside the generation. The full-fidelity pass provides a scheduled correction for candidates that already look competitive under the cheaper estimate.

Cohort Replacement and Rewards. The island then forms a cohort from incumbents and offspring and applies the two-step cohort rule of Section 1.3.5. Let

$$\mathcal{C}_i^{(t)} \;=\; \mathcal{P}_i^{(t)} \cup \mathcal{O}_i^{(t)}$$

denote this combined set. The gate first removes candidates whose structural metric is unacceptable, then ranks the survivors with the task-facing terms defined by the surrogate metric.

The next population $\mathcal{P}_i^{(t+1)}$ is filled from this gated ranking after elite preservation. The same ranked cohort also supplies candidates for the archives described in Section 1.6.7.

The same gated score also defines the reward signal returned to the bandit. If operator o produced offspring c from parent c^{par}, then its base reward is

$$r_o \;=\; \mathbf{1}\big[\Phi^{\mathrm{gate}}(c) < \Phi^{\mathrm{gate}}(c^{\mathrm{par}})\big]\,.$$

A small additive bonus is applied when the offspring also improves the island best $c_i^\star$. The operator-selection rule is therefore rewarded for producing children that survive the same gated ranking used for population replacement.

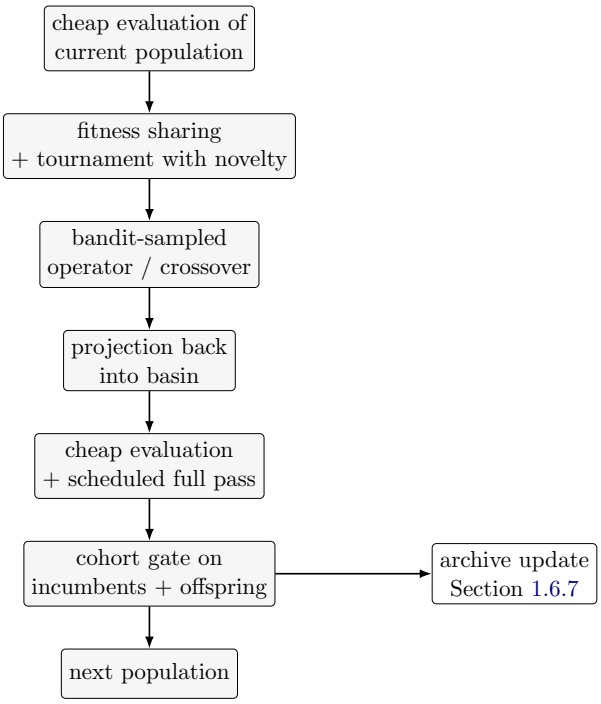
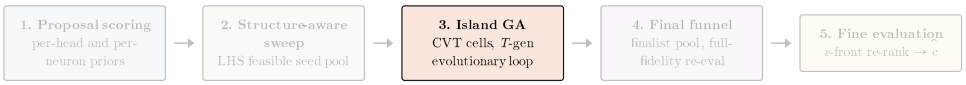


Figure 1.9: One generation of Phase 3 inside a single island. The current population is scored, parents are selected with crowding correction and novelty, offspring are produced and projected back into the basin, the combined cohort is ranked by the two-step cohort rule of Section 1.3.5, and the surviving candidates form the next population. Archive updates use the same ranked cohort and are detailed in Section 1.6.7.

1.6.7 Archives

The generation loop updates the current population, but the search also preserves strong candidates across generations in three global archives. These archives have different purposes: one stores exact elites, one stores the non-dominated cost–distance frontier, and one stores representatives spread over a coarse descriptor grid.



The Hall of Fame, denoted $\mathcal{H}$, is the strict elite memory. It has capacity 50 and is deduplicated by exact mask signature. To prevent the archive from being monopolised by one macro pattern,

it also enforces a per-signature cap of three entries for each macro signature. Here the macro signature is the pair formed by the active-layer set and the rounded head-MAC fraction,

$$\text{macro}(c) \;=\; \big(\{\ell : a_\ell + n_\ell > 0\}, \text{round}(\phi_H(c), 0.1)\big).$$

Candidates enter $\mathcal{H}$ only if they improve on existing members under the gated search score and pass the exact-signature deduplication.

The Pareto archive, denoted $\mathcal{P}$, tracks the non-dominated frontier on the pair (Φ, MAC). A candidate c_a dominates c_b if

$$\Phi(c_a) \le \Phi(c_b), \qquad \text{MAC}(c_a) \le \text{MAC}(c_b),$$

and at least one of these inequalities is strict. The archive stores only non-dominated candidates and is capped at size 100. This archive preserves alternatives that are not globally best in distance but remain competitive at a different point inside the budget band.

The MAP-Elites archive, denoted $\mathcal{M}$, stores one best-distance candidate per cell of a coarse grid on

$$(N_a,\ \text{round}(\phi_H, 0.1)).$$

If a candidate lands in a grid cell already occupied by another candidate, only the lower-distance one is kept. This archive preserves coverage across coarse depth and head-allocation patterns even when those patterns are not currently winning inside the islands.

1.6.8 Final Funnel

Once the generation loop finishes, the search no longer needs to maintain island populations. The remaining task is to gather the strongest candidates produced by the run into one finalist set and re-evaluate them at higher fidelity.

The finalist pool is assembled from the final incumbent, the top candidates of each island, and samples from the three archives. Writing $\widehat{c}^{(T)}$ for the best candidate observed by the end of generation T, the pool is

$$\mathcal{F} \;=\; \{\widehat{c}^{(T)}\} \cup \bigcup_i \text{top}_3(\mathcal{P}_i) \cup \text{sample}_5(\mathcal{H}) \cup \text{sample}_5(\mathcal{P}) \cup \text{sample}_5(\mathcal{M}).$$

The union is then deduplicated by exact mask signature, so every finalist corresponds to a distinct binary architecture.

Each finalist is re-evaluated on the full calibration packs with $R_{\text{final}} = 4$ independent resamples. The final score used in the funnel is

$$\Phi^{\text{final}}(c) \;=\; \frac{1}{R_{\text{final}}} \sum_{r=1}^{R_{\text{final}}} \Phi\Big(c; \mathcal{D}_{\text{cal}}^{\text{full},r}\Big)\,.$$

This final pass is more expensive than the generation-time evaluations, but it is applied only to the small finalist set that survives the funnel.

1.6.9 Fine Evaluation and Selection

The final pass produces a set of high-fidelity distances, but the search does not select the winner by a raw minimum alone. Distances this close are still subject to calibration noise, so the final decision is made from an ϵ-front.

Let

$$d^\star = \min_{c \in \mathcal{F}} \Phi^{\text{final}}(c).$$

The ϵ-front is

$$\mathcal{F}_\epsilon \;=\; \left\{ c \in \mathcal{F} : \Phi^{\text{final}}(c) \le d^\star + \epsilon \right\}, \qquad \epsilon = 0.005.$$

This keeps all finalists whose high-fidelity score is effectively tied with the best observed one.

The candidates in $\mathcal{F}_\epsilon$ are then sorted lexicographically by

$$\big(\Phi^{\text{final}}(c),\ -\text{MAC}(c)\big).$$

The first key prefers smaller final distance. The second key prefers the candidate that uses more of the allowed compute whenever two candidates are indistinguishable in final distance at the ϵ scale. The selected architecture $\widehat{c}$ is the first element of this lexicographic order.

The procedure therefore ends with a single selected architecture $\widehat{c}$ obtained from the budget-constrained search. This completes the search stage itself. The selected mask determines which structures remain active, but it does not by itself adapt the surviving parameters to the compressed architecture. A separate recovery stage is therefore applied under the fixed mask returned by the search.

1.7 Recovery Process

The search returns a fixed pruned architecture $\widehat{c} = (\mathbf{M}^H, \mathbf{M}^N)$. At high sparsity, this architecture usually requires parameter recovery before evaluation: the surviving heads and neurons inherit the teacher parameters, whereas the residual pathway is altered by the removed components. Recovery therefore trains the selected masked student while keeping the architecture fixed.

The recovery objective combines representation-level and prediction-level supervision. These objectives impose distinct constraints on the compressed model. Hidden-state matching aligns the internal trajectory with the teacher computation, whereas label and logit supervision align the final predictive distribution. Under severe compression, the available capacity may be insufficient to optimise both objectives effectively from the initial step. The recovery procedure is therefore staged. The first stage uses intermediate representation matching to stabilise the surviving layers. The second stage shifts the optimisation target to prediction-level distillation and task supervision. This follows the general principle of knowledge distillation (Hinton et al., 2015) and the use of intermediate supervision in compact language models (Jiao et al., 2020).

Figure 1.10 summarises this recovery logic. It places the fixed mask before all optimisation steps, then separates internal representation recovery from final prediction recovery.

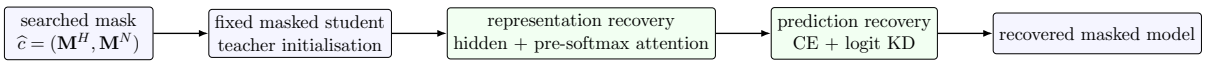


Figure 1.10: Recovery flow after architecture search. The mask is fixed throughout recovery; only the surviving parameters are trained.

The left-to-right order in Figure 1.10 fixes the implementation order used by the recovery process. The student parameters are initialised from the teacher. The head and neuron masks are hardened to binary tensors and are not optimised. Fully dropped layers are bypassed as identities during masked training. The classifier is frozen during the representation stage and unfrozen for prediction recovery. The procedure therefore separates architecture selection from parameter recovery: search chooses the structure, and the recovery process trains the selected structure.

1.7.1 Intermediate Representation Recovery

The first recovery stage trains the masked student before the classifier is allowed to adapt. Its purpose is to restore an internal trajectory that can support subsequent prediction training. If prediction loss is applied immediately, classifier gradients are propagated through representations that may already be poorly aligned with the teacher. Hidden-state supervision gives the surviving layers a direct target and supplies a denser signal than the task label alone.

Let the teacher hidden states be

$$h_0^{(\mathcal{T})}(x), h_1^{(\mathcal{T})}(x), \ldots, h_L^{(\mathcal{T})}(x),$$

where $h_0^{(\mathcal{T})}$ is the embedding output. Let $\mathcal{S}(\widehat{c})$ be the ordered set of active student layers under the selected mask. A fixed set of teacher anchor depths $\mathcal{A}$ is chosen along the teacher depth. The mapping

$$\pi : \mathcal{S}(\widehat{c}) \to \mathcal{A}$$

assigns each active student layer to a teacher anchor according to its relative depth. This mapping is dynamic because the active student layers depend on the searched architecture. A shallow candidate and a deeper candidate therefore use different anchor assignments.

For a batch $\mathcal{B}$ and a padding mask $m_x(p)$, the hidden-state loss is

$$\mathcal{L}_{\text{hid}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a)\in\mathcal{M}} \mathbb{E}_{x\in\mathcal{B}} \left[\frac{\sum_p m_x(p) \left\| h_{s,p}^{(\mathcal{S})}(x;\theta) - h_{a,p}^{(\mathcal{T})}(x) \right\|_2^2}{d \sum_p m_x(p)} \right], \tag{1.28}$$

where $\mathcal{M} = \{(s, \pi(s)) : s \in \mathcal{S}(\widehat{c})\}$. A useful interpretation of Equation 1.30 is that each active student layer is asked to reproduce the token representations of its assigned teacher depth. The padding mask removes padded tokens from the comparison, and the division by $d \sum_p m_x(p)$ makes the squared error comparable across sequence lengths.

Hidden-state matching gives each active layer a target representation after the layer has already combined its attention and feed-forward updates. This is useful, although it does not directly constrain how the surviving attention heads choose token interactions inside the layer. The recovery stage therefore adds pre-softmax attention supervision. This term compares the token-pair scores produced by surviving heads before those scores are converted into probabilities. Let $A_{a,r}^{(\mathcal{T})}(x)$ denote the pre-softmax attention score matrix of teacher anchor layer a and head r, before the row softmax is applied. Let $A_{s,r}^{(\mathcal{S})}(x;\theta)$ be the corresponding student score for a surviving head. With $\mathcal{H}_s$ denoting the active heads in student layer s, the attention-score loss is

$$\mathcal{L}_{\text{attn}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a)\in\mathcal{M}} \mathbb{E}_{x\in\mathcal{B}} \left[\frac{1}{|\mathcal{H}_s|} \sum_{r\in\mathcal{H}_s} \frac{\left\| P_x \left(A_{s,r}^{(\mathcal{S})}(x;\theta) - A_{a,\rho(r)}^{(\mathcal{T})}(x) \right) P_x \right\|_F^2}{\left(\sum_p m_x(p) \right)^2} \right], \tag{1.29}$$

where P_x masks padding positions and ρ maps each surviving student head to its teacher head index. A useful interpretation of Equation 1.31 is that it compares the student and teacher token-pair score tables before softmax. This is important because the softmax compresses score

differences into a probability simplex. When one token already dominates a row, very different pre-softmax scores can produce similar post-softmax probabilities. Matching the scores therefore preserves more information about the attention decision than matching the probabilities alone. The denominator normalises by the number of valid token pairs, so longer sequences do not dominate the loss only because they contain more entries.

The representation-recovery objective is

$$\mathcal{L}_{\text{repr}}(\theta; t) = \lambda_h(t)\mathcal{L}_{\text{hid}}(\theta) + \lambda_a(t)\mathcal{L}_{\text{attn}}(\theta). \tag{1.30}$$

The hidden term is the primary recovery signal. The attention term is assigned a smaller weight because it constrains the mechanism that produces the layer update. The weights may be fixed or scheduled during the stage. The classifier is frozen, so validation checkpoints measure whether the recovered body supports the existing prediction head.

For checkpoint selection, the recovery process also maintains an exponential moving average of the student parameters:

$$\bar{\theta}_t = \mu\bar{\theta}_{t-1} + (1-\mu)\theta_t. \tag{1.31}$$

Parameter averaging reduces short-term optimisation noise and often gives a more stable validation estimate than the instantaneous weights (Polyak and Juditsky, 1992). The checkpoint selected at the end of this stage is therefore chosen from the raw or averaged parameter copy according to development accuracy.

1.7.2 Prediction-Level Distillation

After representation recovery, the masked student has been trained toward the teacher's internal trajectory. The classifier is then unfrozen and the objective shifts to prediction behaviour. This stage uses two signals: the ground-truth task labels and the teacher's pre-softmax logits. The labels anchor the model to the dataset, whereas the teacher logits provide soft class relations that are absent from one-hot labels.

Let $z^{(\mathcal{T})}(x)$ and $z^{(\mathcal{S})}(x;\theta)$ be the teacher and student logits. With label smoothing parameter ϵ, the smoothed target distribution for a class label y is

$$q_\epsilon(c \mid y) = (1-\epsilon)\mathbf{1}[c = y] + \frac{\epsilon}{C}.$$

The task loss is

$$\mathcal{L}_{\text{ce}}(\theta) = -\mathbb{E}_{(x,y)\in\mathcal{D}_{\text{train}}} \sum_{c=1}^{C} q_\epsilon(c \mid y) \log \text{softmax}(z^{(\mathcal{S})}(x;\theta))_c. \tag{1.32}$$

The predictive distillation term uses temperature-scaled teacher logits:

$$\mathcal{L}_{\text{kd}}(\theta) = T^2\, \mathbb{E}_{x\in\mathcal{D}_{\text{train}}}\, \text{KL}\left(\text{softmax}\left(\frac{z^{(\mathcal{T})}(x)}{T}\right) \middle\| \text{softmax}\left(\frac{z^{(\mathcal{S})}(x;\theta)}{T}\right)\right). \tag{1.33}$$

The temperature exposes non-argmax class information, and the factor T^2 keeps the gradient scale comparable as T changes (Hinton et al., 2015). When confidence weighting is used, examples with lower teacher confidence receive a smaller KD weight.

The prediction-recovery objective is a dynamically weighted multi-objective loss:

$$\mathcal{L}_{\text{pred}}(\theta; t) = \alpha(t)\mathcal{L}_{\text{ce}}(\theta) + \big(1-\alpha(t)\big)\mathcal{L}_{\text{kd}}(\theta), \tag{1.34}$$

where $\alpha(t)$ increases slowly over the stage. Early optimisation assigns more weight to KD because the teacher distribution provides smoother gradients while the classifier adapts to the

recovered representations. Later optimisation assigns more weight to CE because the final model is evaluated against task labels rather than teacher predictions.

The optimisation flow is therefore

$$\begin{aligned} \text{fixed mask} &\rightarrow \text{representation recovery} \\ &\rightarrow \text{classifier unfreeze} \\ &\rightarrow \text{prediction recovery} \\ &\rightarrow \text{best raw or EMA checkpoint.} \end{aligned} \tag{1.35}$$

The architecture remains fixed across the entire flow. Recovery trains the already masked architecture; it does not activate or deactivate heads or neurons. The recovered accuracy is therefore determined by the searched mask and the staged distillation procedure.

1.8 Technical Architecture of the Solution

The proposed solution follows a modular technical architecture composed of six main stages: model loading, data preparation, pruning search, candidate evaluation, recovery fine-tuning, and final testing. The pretrained Transformer model is loaded through the Hugging Face Transformers library. The target dataset is tokenized and converted into the input format required by the model.

The pruning stage applies structured binary masks to attention heads and feed-forward neurons. A value of 1 indicates that the unit is preserved. A value of 0 indicates that the unit is removed. For example, if a Transformer layer contains twelve attention heads and the pruning mask preserves four of them, then the remaining eight heads are removed from that layer. Similarly, if the feed-forward block contains 3072 intermediate neurons and the mask preserves 256, then only those 256 neurons remain active during the compressed forward pass.

Candidate architectures are evaluated under a fixed computational budget. The search procedure determines the number of preserved units and their distribution across the model. Two candidates may preserve the same number of attention heads and feed-forward neurons while assigning them to different layers. One candidate may allocate more units to early layers, while another may allocate more units to later layers. These different allocations can produce different evaluation results even when the computational cost is similar.

After the pruning search selects a candidate architecture, the compressed model is recovered through fine-tuning. This step updates the remaining weights after structural removal and restores part of the performance affected by pruning. The final model is evaluated using the metric associated with the selected downstream task.

1.9 Technologies Used

This section presents the main technologies used during the design, implementation, experimentation, and management of the proposed solution. These tools provide the required environment for deep learning development, numerical computation, visualization, experiment execution, version control, and project organization.

1.9.1 Python

Python is an open-source, high-level programming language widely used in software development, scientific computing, data analysis, and artificial intelligence. Its ecosystem provides libraries for model implementation, numerical processing, data handling, and experiment automation. The pruning pipeline, training scripts, evaluation scripts, search procedures, and experiment utilities were implemented in Python Python Software Foundation (2026a,b).

Figure 1.11: Python logo.

1.9.2 Transformers

Transformers is an open-source library developed by Hugging Face. It provides a unified interface for loading pretrained Transformer models, tokenizers, configuration files, and task-specific model heads Wolf et al. (2020); Hugging Face (2026).

The implementation uses this library to load the pretrained encoder model and access its internal components. These components include encoder layers, attention modules, feed-forward submodules, hidden states, attention outputs, configuration parameters, and classification heads. Structured pruning requires access to these components because the method removes complete computational units from the model.

Pruning an attention head requires identifying the corresponding projection dimensions inside the attention module. Pruning feed-forward neurons requires selecting intermediate dimensions inside the two-layer feed-forward network. The Transformers library provides a standardized model structure that supports these operations without requiring a complete reimplementation of the architecture.

Figure 1.12: Hugging Face Transformers logo.

1.9.3 PyTorch

PyTorch is an open-source deep learning framework that provides tensor computation, automatic differentiation, and GPU acceleration. It is commonly used for building, training, and evaluating neural networks. The implementation relies on PyTorch to execute forward passes, compute losses, perform backpropagation, apply masks, update model weights, and run recovery fine-tuning PyTorch Foundation (2026a,b).

PyTorch is also used to represent pruning masks as tensors. An attention-head mask can be represented as a binary tensor of shape 12×12 for a model with twelve layers and twelve heads per layer. A feed-forward mask can be represented as a binary tensor of shape 12×3072 for twelve layers and 3072 intermediate neurons per layer.

Figure 1.13: PyTorch logo.

1.9.4 NumPy

NumPy is a fundamental Python library for numerical and scientific computing. It provides efficient multidimensional arrays, mathematical operations, random number generation, and linear algebra utilities. NumPy is used for candidate representation, statistical calculations, random sampling, sorting operations, and search-related utilities NumPy Developers (2026a,b).

Candidate architectures can be stored as arrays indicating the number of active heads and neurons per layer. These arrays are then used to calculate sparsity, compare candidates, and enforce computational constraints.

Figure 1.14: NumPy logo.

1.9.5 Matplotlib

Matplotlib is a Python library used to create static, animated, and interactive visualizations. It is used to present experimental results, search behavior, pruning statistics, training curves, and comparisons between candidate architectures Matplotlib Development Team (2026a,b).

Typical visualizations include validation accuracy after recovery, active heads per layer, preserved feed-forward neurons per layer, and the relationship between computational cost and task performance.

Figure 1.15: Matplotlib logo.

1.9.6 Google Drive

Google Drive is a cloud storage service that allows files to be uploaded, stored, shared, and accessed from different devices. It is used to organize project files, store experimental outputs, keep backup copies of important resources, and transfer files between working environments Google Workspace (2026).

Figure 1.16: Google Drive logo.

1.9.7 Git

Git is a free and open-source distributed version control system designed to track changes in source code and support collaborative software development. It is used to track modifications to the implementation, preserve experiment scripts, compare previous versions, and maintain a structured development history Git (2026); Chacon and Straub (2026).

Figure 1.17: Git logo.

1.9.8 GitHub

GitHub is a cloud-based platform built around Git. It allows developers to store, manage, share, and collaborate on code through repositories. It is used to host the project repository, synchronize code between machines, and maintain a remote copy of the implementation GitHub Docs (2026); GitHub (2026).

Figure 1.18: GitHub logo.

1.10 Datasets Used

The experimental evaluation uses supervised natural language understanding tasks. These tasks evaluate sentence classification and sentence-pair reasoning after structured pruning.

The selected benchmarks are SST-2, QNLI, and MNLI from the GLUE benchmark (Wang et al., 2018). The dataset discussion is limited to the required information because the main focus of the project is structured model pruning.

1.10.1 Pre-Training Corpora

The pretrained encoder model was originally trained on BooksCorpus and English Wikipedia (Devlin et al., 2019; Zhu et al., 2015). BooksCorpus provides long narrative text from unpublished books. English Wikipedia provides factual and encyclopaedic text. These corpora provide broad linguistic representations before supervised task adaptation.

The model uses WordPiece tokenisation (Wu et al., 2016). Frequent words are represented as complete tokens, while rare words are decomposed into subword units. For example, a rare morphological form can be split into smaller units instead of being mapped to an unknown token. The same tokenizer is preserved during fine-tuning, pruning, and evaluation.

1.10.2 Fine-Tuning and Evaluation Benchmarks

Table 1.3 presents the selected evaluation tasks, including their input format, label space, and representative labelled examples.

Task	Example Input	Possible Labels	Correct Label
SST-2	Sentence: *"The movie is beautifully acted and emotionally powerful."*	`positive`, `negative`	`positive`
SST-2	Sentence: *"The story is dull, slow, and completely forgettable."*	`positive`, `negative`	`negative`
QNLI	Question: *"Where is the Eiffel Tower located?"* Sentence: *"The Eiffel Tower is located in Paris, France."*	`entailment`, `not_entailment`	`entailment`
QNLI	Question: *"Where is the Eiffel Tower located?"* Sentence: *"The Eiffel Tower was completed in 1889."*	`entailment`, `not_entailment`	`not_entailment`
MNLI	Premise: *"A man is playing a guitar on stage."* Hypothesis: *"A person is performing music."*	`entailment`, `contradiction`, `neutral`	`entailment`
MNLI	Premise: *"A man is playing a guitar on stage."* Hypothesis: *"No one is playing an instrument."*	`entailment`, `contradiction`, `neutral`	`contradiction`
MNLI	Premise: *"A man is playing a guitar on stage."* Hypothesis: *"The man is performing in a crowded theatre."*	`entailment`, `contradiction`, `neutral`	`neutral`

Table 1.3: Input formats and label spaces for the selected evaluation tasks.

SST-2

SST-2 is a binary sentiment classification task derived from the Stanford Sentiment Treebank (Socher et al., 2013). Each input is a single sentence, and the model predicts whether its sentiment is `positive` or `negative`.

This task provides a sentence-level evaluation of the compressed model. A model must distinguish positive expressions such as *"excellent and moving"* from negative expressions such as *"boring and weak"*. Accuracy is used as the evaluation metric.

QNLI

QNLI is derived from the Stanford Question Answering Dataset (Rajpurkar et al., 2016). It is formulated as a sentence-pair classification task. The input contains a question and a candidate sentence. The model predicts one of two labels: `entailment` or `not_entailment`.

A label of `entailment` indicates that the sentence contains the answer to the question. A label of `not_entailment` indicates that the sentence does not provide the requested information. For example, if the question asks where the Eiffel Tower is located, the sentence *"The Eiffel Tower is located in Paris, France"* receives the label `entailment`. The sentence *"The Eiffel Tower was completed in 1889"* receives the label `not_entailment`.

This task evaluates the ability of the compressed model to compare two text segments. Accuracy is used as the evaluation metric.

MNLI

MNLI is a natural language inference task (Williams et al., 2018a). Each input contains a premise and a hypothesis. The model predicts one of three labels: `entailment`, `contradiction`, or `neutral`.

A label of `entailment` indicates that the hypothesis follows from the premise. A label of `contradiction` indicates that the hypothesis is incompatible with the premise. A label of `neutral` indicates that the hypothesis may be possible without being guaranteed by the premise.

For example, given the premise *"A man is playing a guitar on stage"*, the hypothesis *"A person is performing music"* receives the label `entailment`. The hypothesis *"No one is playing an instrument"* receives the label `contradiction`. The hypothesis *"The man is performing in a crowded theatre"* receives the label `neutral`, since the premise does not specify the audience or the exact venue.

MNLI provides a stronger sentence-pair reasoning evaluation than single-sentence sentiment classification. Accuracy is used as the reported metric.

1.11 Model

1.11.1 BERT

BERT is an encoder-only Transformer architecture trained with masked language modelling and next-sentence prediction (Devlin et al., 2019; Vaswani et al., 2017). Its bidirectional self-attention allows each token representation to incorporate information from both left and right context.

$\text{BERT}_{\text{BASE}}$ contains twelve Transformer encoder layers. Each layer has twelve attention heads and a feed-forward block with intermediate width 3072. The hidden size is 768, and the full model contains approximately 110 million parameters. For classification tasks, the representation of the `[CLS]` token is passed to a task-specific classifier.

The pruning procedure targets structured units inside the encoder blocks. Attention heads and feed-forward neurons are removed as complete units. This pruning form supports practical acceleration because it can reduce matrix dimensions and remove complete computational paths.

For example, if layer 3 preserves only 2 attention heads out of 12, then the attention computation in that layer is reduced. If the same layer preserves only 192 feed-forward neurons out of 3072, then the intermediate feed-forward projection is also reduced. The final compressed architecture is therefore described by its total sparsity and by the distribution of preserved units across layers.

1.12 Experimental Environment

All experiments are conducted on a laptop equipped with an NVIDIA RTX 4090 Laptop GPU with 16 GB of VRAM. Model loading, tokenization, fine-tuning, pruning, checkpoint handling, and evaluation are implemented using the Hugging Face Transformers library (Wolf et al., 2020).

Mixed precision training is used when numerically stable (Micikevicius et al., 2018). It reduces memory consumption and can improve throughput on compatible hardware. Numerical stability is monitored during recovery fine-tuning, especially for highly compressed architectures.

The hardware environment imposes practical constraints on the experimental protocol. Laptop GPUs are affected by thermal limits, power configuration, driver behavior, and cooling conditions. Therefore, wall-clock measurements are hardware-dependent. Small accuracy differences are interpreted cautiously because repeated runs may vary due to random initialization, data ordering, dropout, and GPU-level nondeterminism.

1.12.1 Baselines and Reporting

The comparison includes the proposed search method, random search, beam search, and structured pruning references from CoFi (Xia et al., 2022) and ZipLM (Kurtic et al., 2023). Comparisons are reported under matching constraints whenever possible, especially the target computational budget, recovery procedure, and evaluation metric.

The reported quantities include development accuracy, giga operations at reference sequence length 128, compute keep ratio, number of active attention heads, number of active feed-forward neurons, and number of active layers. Very small accuracy differences are treated cautiously because they may fall within normal variation across repeated runs.

1.13 Conclusion

This chapter defined the complete implementation pipeline used in this work. The method begins with a binary structural parameterisation of the compressed architecture, then constrains the candidate space with a MAC-based budget model. Candidate masks are compared before recovery through a calibration-based surrogate metric, and the selected mask is produced by a genetic search procedure that maintains diversity across different architectural regions. The selected architecture is then recovered through staged distillation while keeping the mask fixed.

The main design choice is the separation between architecture selection and parameter recovery. The search stage decides where the remaining computation is placed. The recovery stage then adapts only the surviving parameters of that selected architecture. This separation makes the method reproducible: the mask representation, cost model, ranking metric, search procedure, and recovery process are each specified before the results are reported.

The next chapter reports the observed accuracy of the selected models and compares the proposed search procedure with simpler search baselines and structured pruning methods from the literature.

Results (Under Work)

2.1 Result Tables and Discussion

This section reports the development-set results obtained under the experimental setting of Section 1.12. The first comparison isolates the effect of the search procedure by holding the teacher, recovery process, and target budget fixed while changing only the architecture-selection strategy. The second comparison places the recovered models against structured pruning baselines from the literature and against the uncompressed task baseline.

In addition to accuracy, the procedure has a measurable runtime cost. Under the hardware setting of Section 1.12, end-to-end runs on QNLI and SST-2 required nearly six hours on average, whereas MNLI required about thirty hours on average. This difference is consistent with the larger training split and heavier recovery cost of MNLI.

2.1.1 Search Baselines

The first question is whether the evolutionary search improves on simpler budget-constrained selection rules. Table 2.1 compares the proposed method against random search and beam search under matched recovery and matched budget targets. The comparison is organised by the final recovered model size, since the search problem changes materially between the tiny and moderate regimes.

Table 2.1: Development accuracy under matched recovery for three search strategies. Random search and beam search use the same teacher, budget band, and recovery procedure as the proposed method.

Task	Final model	Params	Random	Beam	Ours
QNLI	tiny	3.0M	84.72	85.36	**88.20**
QNLI	moderate	38.0M	90.74	91.10	**91.90**
MNLI	tiny	2.5M	79.58	80.05	**80.70**
MNLI	tiny	3.0M	80.18	80.76	**81.40**
MNLI	moderate	38.0M	84.92	85.19	**85.60**
SST-2	tiny	3.3M	91.21	91.76	**92.43**
SST-2	moderate	39.0M	93.08	93.31	**93.70**

The gap is largest in the tiny regime. On QNLI, the proposed search reaches 88.2 at 3.0M parameters, whereas beam search reaches 85.36 and random search remains lower still. The same pattern holds on MNLI and SST-2: once the budget forces the architecture into a very small parameter regime, search quality has a direct effect on recoverability. In the moderate regime the gap narrows, but the proposed method still remains ahead. This is consistent with the design of the search procedure. When many feasible candidates have similar cost, random and beam methods can still find workable masks, but they do not preserve the same diversity of structural allocations as the island-based evolutionary search.

2.1.2 Comparison to Structured Pruning Baselines

The second question is whether the recovered models remain competitive against established structured pruning methods at comparable sizes. Table 2.2 compares the recovered models against representative CoFi and ZipLM results, together with the task-fine-tuned uncompressed baseline used in this study.

Table 2.2: Comparison against structured pruning baselines and the uncompressed task baseline. Scores are development accuracy.

Task	Model	Params	Score	Note
QNLI	CoFi, 95%	5.0M	86.1	high compression
	ZipLM, near-size	3.2M	87.6	13× speedup
	Ours	**3.0M**	**88.2**	best tiny model
	CoFi, 60%	34.0M	91.8	moderate compression
	ZipLM, 2×	38.0M	91.4	same size as ours
	Ours	**38.0M**	**91.9**	above baseline
	task baseline	full	91.4	uncompressed run
MNLI	CoFi, 95%	5.0M	80.6	high compression
	ZipLM, near-size	3.5M	81.3	13× speedup
	Ours	**3.0M**	**81.4**	above near-size baseline
	Ours	**2.5M**	**80.7**	smaller variant
	CoFi, 60%	34.0M	85.3	moderate compression
	ZipLM, 2×	38.5M	84.8	close size
	Ours	**38.0M**	**85.6**	above both
	task baseline	full	84.7	uncompressed run
SST-2	CoFi, 95%	5.0M	90.4	high compression
	ZipLM, near-size	3.6M	91.7	14× speedup
	Ours	**3.3M**	**92.43**	strongest tiny model
	CoFi, 60%	34.0M	93.0	moderate compression
	ZipLM, 2×	38.7M	93.4	close size
	Ours	**39.0M**	**93.7**	best moderate model
	task baseline	full	93.22	uncompressed run

Three patterns stand out. First, the proposed method is strongest in the smallest models. The gains over CoFi and ZipLM are most visible in the 3M to 3.3M range, where the search and recovery stages must preserve only a very small surviving computation. Second, the moderate-compression models remain competitive even though that regime is less forgiving to search errors: the 38M QNLI and MNLI models recover to 91.9 and 85.6, and the 39M SST-2 model reaches 93.7. Third, the recovered models approach or slightly exceed the uncompressed task baselines on QNLI and SST-2, and improve clearly over the reported structured baselines at comparable size on all three tasks. Taken together, these results support the claim that the main benefit of the method is not only the final pruning ratio, but the ability to preserve recoverable architectures under severe structural compression.

2.2 Conclusion

The results show that the proposed search procedure improves over the internal random and beam-search baselines under the same recovery setting. The improvement is clearest in the smallest models, where the available computation is limited and the placement of the remaining heads and feed-forward neurons has a strong effect on recoverability.

Against structured pruning baselines from the literature, the proposed models remain competitive at both tiny and moderate sizes. On QNLI, the 3.0M-parameter model reaches 88.2 accuracy and the 38.0M-parameter model reaches 91.9. On MNLI, the 3.0M and 38.0M models reach 81.4 and 85.6, respectively. On SST-2, the 3.3M model reaches 92.43 and the 39.0M model reaches 93.7.

The runtime cost is also material. QNLI and SST-2 runs take roughly six hours on average, while MNLI takes about thirty hours because of its larger training split and heavier recovery phase. The results therefore support the usefulness of the search and recovery pipeline, while also showing that the method trades additional search time for stronger recovered sparse architectures.

Bibliography

Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM J. Comput.*, 32(1):48–77, 2002.

Erick Cantú-Paz. *Efficient and Accurate Parallel Genetic Algorithms*. Kluwer Academic Publishers, 2000.

Scott Chacon and Ben Straub. Pro git: About version control. https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control, 2026. Accessed: 2026-05-22.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

Qiang Du, Vance Faber, and Max Gunzburger. Centroidal Voronoi tessellations: Applications and algorithms. *SIAM Review*, 41(4):637–676, 1999.

Git. Git. https://git-scm.com/, 2026. Accessed: 2026-05-22.

GitHub. Github: Developer platform. https://github.com/, 2026. Accessed: 2026-05-22.

GitHub Docs. About github and git. https://docs.github.com/en/get-started/start-your-journey/about-github-and-git, 2026. Accessed: 2026-05-22.

Google Workspace. Google drive: Share files online with secure cloud storage. https://workspace.google.com/products/drive/, 2026. Accessed: 2026-05-22.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NeurIPS Deep Learning Workshop*, 2015.

Hugging Face. Transformers documentation. https://huggingface.co/docs/transformers/index, 2026. Accessed: 2026-05-22.

Xiaoqi Jiao et al. TinyBERT: Distilling BERT for natural language understanding. In *Findings of EMNLP*, 2020.

Eldar Kurtic, Elias Frantar, and Dan Alistarh. ZipLM: Inference-aware structured pruning of language models. In *NeurIPS*, 2023.

Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.

Matplotlib Development Team. Matplotlib documentation. https://matplotlib.org/stable/index.html, 2026a. Accessed: 2026-05-22.

Matplotlib Development Team. Matplotlib: Visualization with python. https://matplotlib.org/, 2026b. Accessed: 2026-05-22.

Michael D. McKay, Richard J. Beckman, and William J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979. doi: 10.1080/00401706.1979.10489755.

Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *NeurIPS*, 2019.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2018.

Gergely Neu. Explore no more: Improved high-probability regret bounds for non-stochastic bandits. In *NeurIPS*, 2015.

NumPy Developers. Numpy. https://numpy.org/, 2026a. Accessed: 2026-05-22.

NumPy Developers. Numpy: The fundamental package for scientific computing with python. https://pypi.org/project/numpy/, 2026b. Accessed: 2026-05-22.

Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

Python Software Foundation. About python. https://www.python.org/about/, 2026a. Accessed: 2026-05-22.

Python Software Foundation. The python tutorial. https://docs.python.org/3/tutorial/, 2026b. Accessed: 2026-05-22.

PyTorch Foundation. Pytorch documentation. https://docs.pytorch.org/docs/stable/index.html, 2026a. Accessed: 2026-05-22.

PyTorch Foundation. Pytorch project. https://pytorch.org/projects/pytorch/, 2026b. Accessed: 2026-05-22.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392. Association for Computational Linguistics, 2016. doi: 10.18653/v1/D16-1264.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725. Association for Computational Linguistics, 2016. doi: 10.18653/v1/P16-1162.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642. Association for Computational Linguistics, 2013.

Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *EMNLP Workshop BlackboxNLP*, 2018.

Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1112–1122. Association for Computational Linguistics, 2018a. doi: 10.18653/v1/N18-1101.

Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL-HLT*, 2018b.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020. URL https://aclanthology.org/2020.emnlp-demos.6/.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Mengzhou Xia, Zexuan Zhong, and Danqi Chen. Structured pruning learns compact and accurate models. In *ACL*, 2022.

Yukun Zhu, Ryan Kiros, Richard Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 19–27, 2015.